

The Software Factory

The concepts, and the run sheet for every session that teaches them. Part one explains each idea once. Part two is what a facilitator reads standing up, and it points back at part one by name.

Built by `make book`, from the same pages the browser serves

Checked by its own text layer, read back with `pdftotext` after every build

Part one, the concepts

- | | |
|----|--|
| 01 | What a factory is, and why a better prompt is not one
Overview |
| 02 | Boundary
Layer one |
| 03 | Context and skills
Layers two and three |
| 04 | Execution
Layer four |
| 05 | Verification
Layer five |
| 06 | Self-healing
Layer five, continued |
| 07 | Delivery
Layer six |
| 08 | Who does what, under each shape
The same factory, four companies |
| 09 | A rule written in a prompt is a request, and a rule written in a gate is a limit
Measured, not asserted |
| 10 | What the field has measured, and what it says about this architecture
Grounding |
| 11 | The patterns that outlast both products
What survives the products |
| 12 | On a platform |

The same architecture, bought instead of built

Part two, the sessions

- | | |
|----|---|
| 01 | Design a Software Factory for Your Coding Agents
lesson, 60 minutes |
| 02 | Build Your First Software Factory Execution Harness
lesson, 60 minutes |
| 03 | Enforce Code Quality with Automated Agent Verifiers
lesson, 60 minutes |
| 04 | Automate Pull Requests with Background Coding Agents
lesson, 60 minutes |
| 05 | Design Your Software Factory Architecture
module, 120 minutes |
| 06 | Isolate Agent Work with an Execution Harness
module, 120 minutes |
| 07 | Route the Right Context to Every Task
module, 120 minutes |
| 08 | Make Agents Prove Their Work
module, 120 minutes |
| 09 | Build Self-Healing Engineering Loops
module, 120 minutes |
| 10 | Orchestrate and Ship Verified PRs
module, 120 minutes |
| 11 | Design an AI-Native Software Factory for Your Organization
workshop, 360 minutes |
| 12 | Office hours and capstone demo day
office-hours, 60 minutes |

What a factory is, and why a better prompt is not one

The six layers six-layers

A coding agent on its own is a generator. Around it sit six jobs that decide whether its output is worth your attention. Boundary says what may be attempted. Context says what the task gets to know. Skills hold the know-how that is reused rather than retyped. Execution says where the work happens and what it can reach. Verification says what proof must exist. Delivery says how finished work reaches a person and who decides it ships.

Naming them separately matters because they fail separately. Work that keeps arriving wrong is usually a context problem. Work that arrives plausible but broken is a verification problem. Work that damages something is a boundary problem. One word for all three leaves you tuning a prompt when the missing piece is a gate.

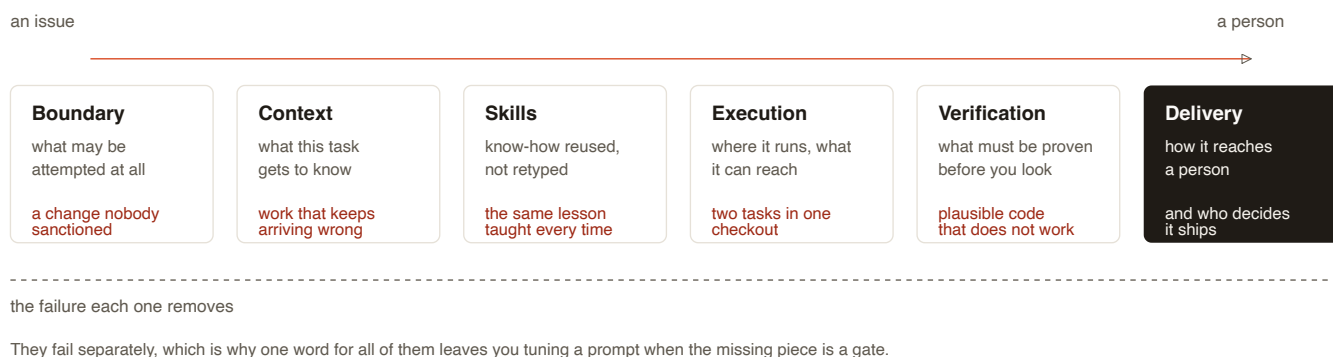


Figure 1. The six jobs around a coding agent. Each box names what it decides; the line beneath it names what goes wrong when it is missing.

The layers are not a maturity model. Most teams already have two or three by accident, written down nowhere. The exercise is finding which one is absent.

Why the prompt keeps growing mega-prompt

The usual response to an agent getting something wrong is to add a sentence to the instructions. It works, so it happens again, and the instructions become a document that every task pays for and no task needs in full. Accuracy falls as the pile grows, because the sentence that matters for this task is now competing with forty that do not apply to it.

Routing replaces piling. Each task receives the repository rules that govern the files it will touch, the skills its kind of work needs, and the task itself. What a task does not need, it never sees.

The executor is an interface executor-interface

Nothing in the factory knows which agent runs the work. An executor is any command that

accepts a prompt on standard input, does something to a working directory, and exits. That indirection buys three things: the loop can be tested against a recorded transcript with no model and no network, a different agent can be swapped in without touching policy, and the same task can be replayed to compare two agents on identical input.

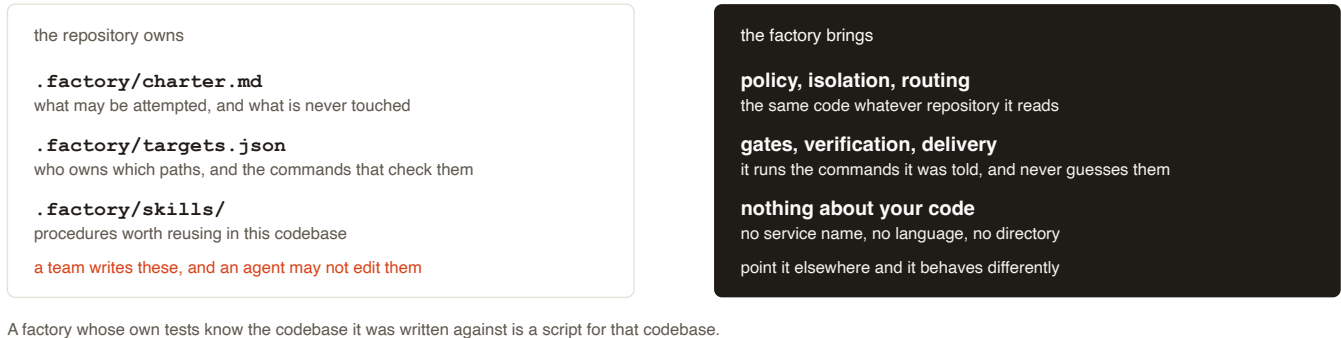


Figure 6. The split that makes a factory portable. Facts on the left, logic on the right, and the line between them is what a second repository tests.

It is also the honest boundary of the claim. The factory is not a model. It is the system that decides what a model is allowed to attempt and what happens to the result.

Run `make check` and no model is called. The same loop, the same policy, the same gates, a recorded agent.

Layer one

Boundary: what may be attempted without asking

Boundary boundary

Boundary is the answer to a question every team answers informally and inconsistently: which work can an agent finish while nobody is watching. Left implicit, the answer drifts by person and by day, and the first surprise is expensive. Written down, it becomes something a run can read before it starts, and something a reviewer can point at afterwards.

The charter charter

One file, owned by a person, that the factory reads before every run. It states the tier, the paths that are load-bearing, the kinds of task that may proceed unattended, what counts as finished, and the conditions under which the factory stops producing. It is short on purpose. A policy nobody rereads is a policy that stops describing the team.

The agent may not edit it. A proposed change to the charter is a pull request a human reads like any other, which is the only way the limits stay meaningful.

Autonomy tracks consequence autonomy-tier

The instinct is to grant freedom by difficulty: small changes unattended, large ones reviewed. That is the wrong axis. A large migration on a project with no users is recoverable. A small edit to the code that decides what a customer is charged is not. Freedom should follow how bad the worst outcome is and how easily it is undone.

the same work, four operating models

	solo	startup	scaleup	enterprise
runs start themselves				
plans approve themselves				
the money path is off lim.				
the contract needs a plan				
the queue stops at	1 waiting	3 waiting	5 waiting	2 waiting
the money item	a pull request, unattended	a plan, waiting on a person	refused, routed to its team	refused at the boundary

One repository, one command to switch. The room sees the same item open a pull request in one shape and stop dead in another inside a minute.

Figure 5. Read from `profiles/*/board.json`, `targets.json` and each charter, so the slide and make `profile` cannot disagree. Nothing about the codebase changes between these columns; only what a person insists on seeing.

So the tier is a property of the repository, and it is refined per area. In this lab the budget engine is always human-approved whatever the size of the change, and the console can be changed unattended even when the change is large.

Protected paths protected-path

A list of globs the factory will not modify on its own. Credentials and configuration, the money path, anything that governs the factory itself, and the tests that define existing behaviour. A task that needs one of them is not refused forever; it is routed to a person who decides.

The useful property is that a refusal is visible. A run that stops with a reason is information. A run that quietly rewrites an assertion so its own work passes is the failure this list exists to prevent.

A protected path nobody has watched refuse a task is a comment. The lab keeps a task that must be refused, and scores the refusal.

Back-pressure is a number queue-cap

The constraint on a factory is not how many agents can run. It is how many decisions are waiting on a person. Generation is cheap and getting cheaper; the attention needed to understand a change, integrate it, and take responsibility for it is fixed.

So the charter carries a cap on how many items may sit awaiting review, and a run refuses to start work when the queue is full. Without it the first productive afternoon ends with more pull requests than anyone will read, which is indistinguishable from having built nothing.

Every path has one owner ownership-graph

Policy stated over globs alone gets vague as a repository grows. A graph of targets, each owning a set of paths and naming the commands that check it, makes the policy specific: this area belongs to this target, the target has this autonomy, and these are the checks that must pass for it.

It also makes a gap loud. A directory that no target claims is a directory with no owner, no checks and no autonomy rule, and the build fails rather than guessing.

Context and skills: what the task is told

Context context

Context is everything the task needs to know that is not the task. Where the code lives, how this repository does things, which invariants are not negotiable, what the surrounding code already looks like. Agents rarely fail for want of intelligence on a familiar codebase; they fail because nobody told them the thing everyone on the team already knows.

Skills skills

A skill is a procedure worth reusing: how to add a migration here, how to write a test that will still make sense in a year, what a review looks for. It says what good work is and what evidence proves it, and leaves the mechanics to the agent. Kept separately from any one prompt, it improves once and every task gets the improvement.

The distinction from context is worth holding. Context describes this repository. A skill describes a way of working, and often survives being copied to a different repository unchanged.

The context router context-router

A small piece of code that turns a task into the exact material that task should receive. It reads which files the work is expected to touch, asks the ownership graph which targets those belong to, and assembles the rules, the skills and the task description for that combination.

Being code rather than prose, it can be tested. Give it a task and assert what it routed. That is the difference between believing the right context arrived and knowing it did.

A skill states the outcome, not the keystrokes reusable-skill

The failure mode when writing one is to script every move, which produces something long, brittle, and worse than the agent's own judgement. The version that holds up names the outcome, the constraints that are not obvious, and the evidence required at the end.

Only what the change touches affected-targets

Running every check in a repository of any size is slow enough that people stop doing it. Deriving the affected set from the diff and the dependency edges keeps the loop fast, and it is the same calculation that tells the router which rules a task needs. One graph, two uses.

The edge that catches people is the shared one. A change to a contract that three services are built against affects all three, and a naive file-to-target mapping will run one suite, pass, and ship a break.

Execution: where the work happens

Execution execution

Giving an agent your checkout is the default because it is the path of least setup, and it is the reason most people stop trusting background work. Two tasks collide. A half-finished experiment gets committed. You switch branches under a run and it writes into the wrong one. None of that is the agent behaving badly; it is the absence of a place for it to work.

A worktree per task worktree

Git already solves this. A worktree is a second checkout of the same repository at a different commit, sharing one object store, created in about a second and removed as easily. Each task gets one, does its work there, and it is thrown away when the task ends.

The properties that matter: your own branch is never touched, two tasks cannot interfere, a failed run leaves an inspectable directory rather than a mess in your working copy, and cleanup is a single command rather than a careful sequence.

The branch is the claim branch-claim

When more than one run can start for the same item, something has to decide which one owns it. A lock table is a service to operate. A deterministic branch name pushed to the shared remote is not: the first push wins because the remote rejects the second, and the loser stops.

This is worth dwelling on when teaching, because it is the moment the design stops being a script and starts being a system. Coordination came from a tool everyone already runs, and there is nothing extra to keep alive.

Limits on files, commands and network sandbox-limits

Isolation answers where the work happens. Limits answer what it can reach from there. Which paths may be written, which commands may run, whether anything may leave the machine. Stated per task rather than globally, because the task that regenerates a lockfile legitimately needs the network and the task that edits a component does not.

Two rules keep this from becoming theatre. The limits are declared where a person can read them, not buried in the agent's instructions where they are a suggestion. And exceeding them ends the run rather than printing a warning nobody sees.

Verification: what must be proven before you look

Verification verification

Agents are good at producing code that reads correctly. Review is where that becomes expensive, because plausible wrong code costs more attention than obviously wrong code. Verification is everything that happens before a person spends any attention at all, and its job is to make sure what arrives has already survived every check a machine can run.

The gate script owns the verdict gate-verdict

Checks run in one script that ends with a single line stating the result. Nothing is allowed to paraphrase it. An agent reporting that everything passed is a claim; the line is evidence, and the two are not interchangeable.

The behaviour that earns its keep is what happens when a required check is missing. A repository with no test command must not produce a green run. It produces a misconfigured verdict and stops, because silence and success are indistinguishable otherwise and only one of them is safe.

Holdout tests holdout-test

A test the implementing agent never sees cannot be optimised against. When the agent writes the tests and then makes them pass, a green suite means the code agrees with itself. Keeping some behavioural checks out of reach turns passing back into evidence about behaviour.

They are not a replacement for tests written alongside the change. They are the independent sample, and a handful covering the contract is worth more than a large suite the agent could read.

The writer never grades the work cold-verifier

Verification is delegated to a second agent that receives the diff and the task, and explicitly not the account of how the work went. Told what was attempted, a reviewer anchors on it and starts confirming rather than checking.

What this catches in practice is not syntax. It is the change that passes its tests while doing something the task never asked for: widening what gets stored, altering an unrelated behaviour, weakening an assertion to make a failure go away.

Revert the fix and watch the test fail negative-proof

The strongest check available, and no gate can make it. Take the new test, undo the change it was written for, and confirm it goes red. If it stays green the test proves nothing, and the run says so rather than shipping a passing suite that was never connected to the behaviour.

This one line of reasoning is the most useful thing in the whole system for most teams, and it applies with no agents involved at all.

Code decides what a verdict may claim evidence-rule

After the agent has finished talking, ordinary code reads the artifacts the run produced and decides what the result is allowed to say. A pass whose test output cannot be found is downgraded. A recommendation with no citation of the thing it rests on is downgraded. The agent's description of the run is never consulted for any of this.

Let the agent investigate freely, then let deterministic code decide what it is permitted to assert. Freedom where judgement helps, and no freedom at all in the sentence a person will act on.

One definition, checked from every side contract-drift

When a type is restated in three languages, the three restatements disagree eventually, and the disagreement is found in production. Keeping one definition and asserting each implementation against it turns that into a test failure at the moment of the change.

For a factory it does something more specific. It gives the cross-stack task a real shape: edit the shared definition and three suites go red at once, in three languages, which no single-language demonstration can show.

Layer
five,
continued

Self-healing: failing usefully, and stopping

Doer and Tester doer-tester

Two roles rather than one. The Doer implements. The Tester decides whether the result meets the task, and when it does not, hands back the reason attached to the failure rather than a bare rejection. The next attempt starts from that reason.

Splitting the roles is what makes the second attempt cheap. One agent grading itself has already spent its reasoning on the version it produced, and tends to defend it.

A loop that cannot end is not a loop exit-criteria

Every cycle needs three declared endings: it passed, it ran out of attempts, or it exceeded its time. Without the second and third, a task that cannot be done consumes attempts until somebody notices, and what somebody notices first is the bill.

Escalation is a real outcome, not a failure of the design. The item goes back to a person with the attempts recorded and the reason the loop gave up, which is more useful than a run that is still going.

Measure where the time actually went latency-budget

The surprise on a first real run is that generation is rarely the slow part. Time accumulates in setup, in repeated checks, in verification that replays work already done, and in the stretch where the run is waiting on a person and nothing says so.

So each stage is timed separately and recorded: claiming the item, assembling context, implementing, gates, verification, and human wait. Every figure quoted in these pages is read from that record by a test, which is why none of them can drift from the runs that produced them.

The button that opens a session is not always called the same thing. A resting card offers Investigate, a card the factory is working offers Build, and Open session appears once a session exists and is idle. Follow the card, not the label: whichever button is on it is the one that opens what the run is doing. A run sheet that named a fixed button sent a presenter hunting for one that was not there.

one run, every stage timed separately



Generation is rarely the slow part. Waiting on a person usually is, and it stays invisible until somebody measures it.

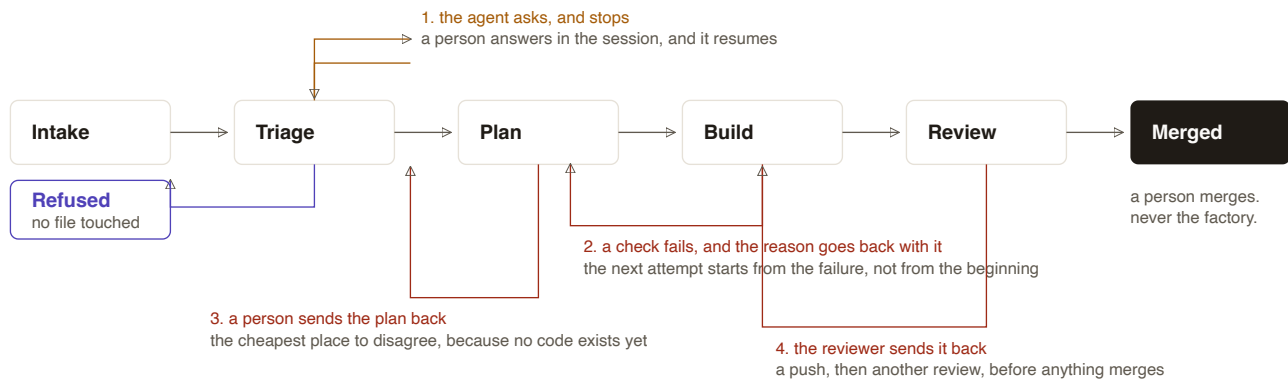
Figure 3. A run with each stage timed on its own, drawn from `evidence/factory-run.json` on `openrouter/deepseek/deepseek-v4-flash`. Every figure here is the median of what the factory recorded, so none of them can drift from what actually happened.

Without per-stage numbers the only available response to a slow factory is to make the model faster, which is the one thing you do not control.

Layer six

Delivery: how finished work reaches a person

Delivery delivery



Four ways work comes back and one way it stops. A demonstration that only shows the straight line through the middle is not a demonstration of a factory.

Figure 4. The straight line across the top is the path everyone expects. The loops below it are where the engineering is, and the box on the left is the outcome that looks like nothing happened.

The last layer is the one most demonstrations skip, and it is where trust is actually built or lost. Work has to arrive somewhere a person already looks, carrying enough evidence to be judged, and the decision to ship has to stay with someone accountable for it.

The queue is somewhere you already look work-queue

State lives in the issue tracker: labels for the stage, a structured comment for the handoff between stages, a branch for the claim. No database, no dashboard to keep alive, and nothing lost when a session ends.

The reason is durability rather than convenience. A run that dies halfway leaves its state in a place the next run and a curious human can both read, and neither of them needs the conversation that produced it.

A pull request that arrives with its evidence draft-pr

The unit of delivery is a draft pull request carrying what happened: the gate verdict, the independent verdict, the negative proof, the stage timings and what it cost. A reviewer opens one thing and can see whether the work was checked before deciding whether it is right.

Draft is deliberate. It signals that a machine produced this and a person has not yet stood behind it, which is exactly the state it is in.

Nothing merges itself `human-merge-gate`

No tier, no confidence score and no run of green results changes this. The pipeline stops at a reviewed pull request and a person decides. The hooks in the repository catch the obvious routes, and the enforcement that counts is a branch protection rule, because a guardrail an agent can edit is not a boundary.

This is also the answer to the question every room asks. The factory does not remove the reviewer. It makes sure that by the time the reviewer arrives, the machine has already done everything a machine can do.

The run record `run-record`

One immutable file per run: what was attempted, which files changed, every verdict, the timings, the cost. Appended to, never rewritten, because the value is in comparing the tenth run to the first.

It is what turns an opinion about whether this is working into a measurement, and it is what the console in this repository displays.

The same
factory,
four
companies

Who does what, under each shape

The four profiles in `profiles/` are usually read as a table of permissions. That is the least interesting thing about them. What each one really describes is a division of labour: who files the work, who decides it is worth doing, who accepts the approach, who reads the diff, who presses merge, and who gets woken up when it was wrong.

Read the four together and the argument becomes hard to avoid. None of them is more advanced than the others. Each moves the same set of decisions to different people, and each pays for that somewhere.

Solo

One person files, decides, accepts, reads and merges. The factory starts its own runs and approves its own plans, because the only reviewer is the person who will read the diff before merging anyway, and a gate between an author and themselves is ceremony.

The queue cap is one. That number surprises people until they think about it: a solo developer with three things waiting is not three times as productive, they are a bottleneck with a backlog. The constraint worth declaring here is attention, not capacity.

The money path is `build`, which reads as reckless and is not. The gate has moved to merge, where this person reads every line regardless. Moving it earlier would add a step that the same human performs twice.

Startup

A few people file and one of them accepts. Runs still start on arrival, because waiting for a person to press start on an obviously good issue is a tax nobody is paying for. Plans no longer approve themselves.

That single change is the whole shape. A small team shares the cost of being wrong without sharing the context that would have prevented it, so somebody reads the approach before code exists. The money path is `propose` for the same reason.

The queue cap is three. Above that, the person accepting plans becomes the constraint and the factory is generating work faster than the team can agree to it.

Scaleup

Teams own areas. Somebody files, the owning team triages, and a person on that team starts the run deliberately rather than letting arrival trigger it. The money path is `refuse`, because it has an owning team and an item that reaches it belongs in their queue rather than in a plan written by whoever filed it.

This is the first shape where the answer to "who decides this" is a different person from "who noticed it", and most of the friction in a scaling engineering organisation lives in that gap. The ownership graph is where that gets written down instead of argued about on the day.

The queue cap rises to five, because there are more people who can read a plan. It does not rise further, because the reviewers are still individuals with calendars.

Enterprise

Nothing is planned or implemented until a person has accepted the item. The tier is `controlled` rather than `supervised`, and the difference is real: work does not begin on arrival at all.

The money path is refused and protected, so a run stops at the boundary rather than producing a plan nobody may act on. The contract package is refused too. What an agent may do here is investigate, and propose.

The queue cap falls back to two, which reads as a mistake and is not. The constraint is no longer how many plans a team can read; it is how many items a change process can carry at once. A larger cap would queue work against a slower gate and call it throughput.

the same six decisions, four divisions of labour

	solo	startup	scaleup	enterprise
decide it is worth doing	the author	the team	the owning team	a change process
accept the approach	nobody	a teammate	the owning team	a named approver
start the run	arrival	arrival	a person	a person
read the diff	the author	a teammate	the owning team	two people
press merge	a person	a person	a person	a person
carry the pager	the author	whoever is up	the owning team	an operations team

One row never moves. Nobody has built a shape where a machine presses merge, and the four that exist disagree about everything else.

Figure 7. The decisions each shape hands to a different person. The row that does not move is the one worth defending.

A team that cannot say which of these four it is will configure its factory by accident, and the accident will be whichever shape the tool shipped with.

Measured,
not
asserted

A rule written in a prompt is a request, and a rule written in a gate is a limit

Everything above this describes how a factory is supposed to behave. This section describes how one actually behaved, on the runs that produced the figures and the screenshots in this material, and the two are not the same. The gap is not a defect report. It is the most useful thing in the workshop, because it is the difference between the two systems in this repository, and that difference is the whole argument for building the layers by hand before buying them.

The hand-built pipeline calls `decide()` before anything runs. Mastra Factory tells the agent about the same ownership graph in `AGENTS.md` and then dispatches the work. One of those systems consults the rule and the other repeats it. Nothing in the second one is broken or misconfigured: the agent was told, in the file the product reads, and the engine simply has no code that asks the graph a question before writing a file.

A limit that depends on the agent choosing to honour it is a limit you have already lost. The test is not whether the model complied on your run. It is whether any code would have stopped it if it had not.

The same change, put to both systems two-answers

The cleanest evidence in the material comes from the route designed to be boring. An issue asks for a Contributing section in the README. The agent triaged it, planned it, built it and opened a pull request, and the run sheet shows that pull request being approved. Then `01-clean-path.spec.ts` hands the very same list of changed files to `decide()`, reading the very same `.factory/targets.json`, and gets the opposite answer: **REFUSE**, because `README.md` has no owner in the graph and falls through to the `repo` target, which is set to refuse.

Two systems, one repository, one diff, two answers. The model is not the variable. The variable is that one of them calls a function.

Run the same route twice and the agent does not always make the same choice. It advanced its own plan on one run and waited on the next. It opened a pull request on one run and pushed a bare branch on another. This is the part worth sitting with, because it defeats the obvious objection. If the behaviour were reliably wrong you would notice in a week and fix the prompt. It is reliably unreliable, which means your demonstration passes, your pilot passes, and the run that matters is the one nobody watched.

What the runs did not honour findings

Each row below was recorded by the suite while it ran, written to `evidence/prompt-vs-gate.json`, and is held against this page by `tests/findings.test.ts`. That test fails if the page cites a finding no run produced, and it also fails if a run recorded an instruction the model ignored and this page does not mention it, which is the failure mode of anybody writing up their own demonstration.

The last column says how strongly each row is evidenced, because they are not all evidenced the same way and a table that hides that is doing the thing this section exists to warn about.

- **The record** means the counter in `evidence/prompt-vs-gate.json` has watched the claim fail on a run it counted, and the gate will fail this page if the row is removed while that stays true. The same counter holds the runs where it did not fail, so a row can be read as often as rarely rather than only as a headline.
- **PR 544** and **PR 547** are pull requests still open to inspection in the example repository. The first changed `README.md`, a path the graph refuses. The second changed `packages/contracts/**`, a `propose` path, in the same commit range that wrote its own plan to `.artifacts/plans/` and then carried on without anyone reading it. Go and look at them.
- **Seen while building** means it happened once, during the sequences that produced this material, and left no artefact that outlived the next lab reset. Treat those rows as a warning about what the system permits rather than as a measurement of how often it does it. The counter is there so that the next time one recurs, it stops being anecdote.

That distinction is the honest shape of the evidence, and it is worth saying to a room out loud. The strongest rows are the two you can click on. The weakest are the ones you have only my word for, and the reason the counter exists at all is that my word is not the standard this repository holds everything else to.

Route	What the prompt asked for	What the run did	Evidence
clean	a person accepts the plan before code is written	Read from an earlier sequence's stage history, where the agent moved its own item from planning into execute about forty seconds after finishing the plan, and later runs waited. No engine code asks who is allowed to advance a stage, so both are available to it.	seen while building
clean	the work is pushed to the branch the factory named for the issue	On one run the factory created <code>factory/issue-<n></code> and the agent committed to a branch it named itself, leaving the factory's branch with no commits on it. Anything keyed to the convention then sees a started job that produced nothing, while the work sits somewhere nobody is looking. The next run followed the convention exactly.	seen while building
clean	finished work arrives as a pull request, which is what the charter calls done	On one run the build succeeded, the commit was pushed and no pull request was opened. On another, on the same issue, one was opened properly. The suite opens what is missing and records having had to.	seen while building
clean	an agent changes only paths the ownership graph allows it to change	It changed <code>README.md</code> , which the graph refuses. Our own boundary layer, reading the same file, declines the identical diff and says why: <code>autonomy: repo is refuse</code> .	the record, and PR 544
cross-stack	a propose target waits for a person to accept the plan before code is written	On one run the plan was written, nobody accepted it, and the agent opened a pull request modifying <code>packages/contracts/**</code> . This is the row that matters	PR 547

Route	What the prompt asked for	What the run did	Evidence
		most: <code>propose</code> exists precisely to stop that, and nothing in the engine can. On another run it waited, which is luck rather than enforcement.	
question	an underspecified issue is asked about rather than guessed at	It asked nothing. The issue was read as a feature request, the agent chose for itself where the new figure should live, and the issue carries no comment.	<code>the record</code>
question	an unanswered question does not produce a pull request	On one run it built and shipped its guess, with no signal anywhere that a judgment call had been made on someone else's behalf. On another it stopped before pushing anything, having still asked nothing.	<code>seen while building</code>
refused	the refusal names the path it will not touch, on the issue, where a person will see it	The refusal itself held on every run: no branch, no pull request, nothing written to the money path. On one run it said nothing about why, which makes a refusal indistinguishable from a stall. On another it named the path plainly.	<code>seen while building</code>
review-rejection	the pull request label agrees with the verdict the re-review wrote	The re-review commented <code>approve</code> and left the pull request with no verdict label at all. One question, two answers, and only one of them reconciled.	<code>seen while building</code>
plan-revision	reentering a stage runs that stage, and nothing else fails quietly beside it	Asking for planning a second time raised a triage decision too, which failed while the plan succeeded. The board shows a fresh plan and no sign of the failure.	<code>seen while building</code>

What each of these costs, and what to do instead what-to-do

Read the table as four distinct problems rather than one, because they need four different answers and only the first is about autonomy levels.

An advisory level costs you the reason you wrote it down. If `propose` cannot hold work at the boundary, then the ownership graph is documentation and the only real control left is the branch protection rule at the end. The fix is not a firmer sentence in `AGENTS.md`. It is a check that runs against the diff before the pull request can merge, reading the same graph, and refusing paths the graph refuses. That check is the one thing in this material you should port into your own repository first, because it works whatever agent you buy.

A silent refusal costs you the ability to tell deciding from dying. An issue with no branch, no comment and no activity looks the same whether the factory declined it on purpose or fell over. The charter says to record the reason and route the item to a person, and no code makes that happen. Anything that stops should have to say so in a place a person already looks.

A model that cannot tell it is guessing will guess. "Ask when unsure" is the weakest instruction in any prompt, because the failure requires the agent to notice its own uncertainty, and the runs where it did not notice look exactly like the runs where there was nothing to ask. Replace it with a rule that does not depend on noticing: an issue that names no owning service does not reach the build stage.

Two representations of one verdict will disagree. The comment is what the reviewer writes and what a person reads. The label is a projection of it that nothing reconciles, so a pull request that had just been approved read as unlabelled, and anybody filtering a queue by label would not have seen it. Pick which one is the record and derive the other from it.

These four are worth more to the room than a run where everything went right. A clean demonstration teaches that the architecture is plausible. This teaches where it actually gives way, which is the only part anybody will have to deal with on a Tuesday.

What the field has measured, and what it says about this architecture

The argument in this book is that the generator is not the missing piece. That is a claim about the world, so it should be held against what has actually been measured rather than against how a good demo feels.

Adoption raises throughput and instability at the same time adoption-evidence

DORA's 2025 research reports that nine in ten technology professionals now use AI at work (<https://dora.dev/insights/balancing-ai-tensions/>), that around a third of developers report little or no trust in the code it produces, and that higher adoption correlates with more delivery throughput and more delivery instability together.

Their conclusion is the one worth carrying into an architecture review. AI is an amplifier: it magnifies a strong platform and it magnifies a weak one, and the difference between the two outcomes is testing, workflow and the platform underneath, not the model.

Read that finding next to the six layers and it stops being a survey result. Throughput rises because generation got cheap. Instability rises because verification and delivery did not change, and those are the two layers that decide whether work arriving faster also arrives working.

Feeling faster and being faster are different measurements speed-evidence

METR ran a randomised trial in which sixteen experienced open-source developers worked 246 real tasks (<https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>) in repositories they already knew, with AI allowed on half of them. They took 19 percent longer with the tools, and estimated afterwards that they had been about 20 percent faster.

Two honest caveats belong with that number, and stating them is what makes it usable. It measures experienced maintainers on codebases they know well, which is the case least likely to benefit, and it measures tools from early 2025.

What survives both caveats is the gap between the felt result and the measured one. A team deciding how far to go on the strength of how the last demo felt is using the instrument this study showed to be wrong by about forty points.

Each layer removes a named risk, not a vague one risk-classes

OWASP published its Top 10 for Agentic Applications (<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>) in December 2025, covering threats that belong to agents rather than to models. Mapping them onto the layers turns a security conversation into an architecture one.

Layer	The risk class it removes	What does the removing
Boundary	Excessive agency, and privilege abuse	A charter that refuses a path, checked by a function rather than described in a prompt
Context	Memory and context poisoning	Routing, which means a task never sees material that has nothing to do with it
Execution	Unexpected code execution	A worktree, a command list and a network boundary, so the blast radius is a directory
Verification	Goal hijack, and tool misuse	A gate that a run cannot talk its way past, and a reviewer that reads the diff cold
Delivery	Human-agent trust exploitation	A pull request that arrives with its evidence, and a merge only a person can press

Using their names has a practical payoff. A security reviewer at a large company can map this architecture onto a list they already hold, rather than reading an argument assembled from first principles by a workshop.

Routing rather than piling is published practice routing-evidence

Anthropic's engineering guidance on effective context engineering (<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>) names three techniques for work that outgrows a context window: compaction, structured note-taking, and sub-agents that explore in their own window and return a summary.

All three are the same move as the context router here. Decide what this task needs, give it that, and keep everything else out, rather than adding a paragraph to a file every task pays for.

What
survives
the
products

The patterns that outlast both products

Both of the systems in this repository will be replaced. These will not. Each one is written the same way: the problem it answers, the forces pulling against each other, the move, what it costs, and where it shows up in the thing built by hand and in the thing bought off the shelf.

A pattern with no cost listed is a recommendation, and a recommendation is the kind of advice you cannot argue with.

A prompt is not a gate `prompt-is-not-a-gate`

Problem	The rules an agent must follow have to be written down somewhere, and the obvious place is the prompt the agent reads.
Forces	A prompt is the cheapest place to put a rule, it needs no code, and it reads convincingly to whoever wrote it. A gate costs work and has to be maintained. Meanwhile the agent complies often enough that the difference stays invisible until the run that matters.
The move	Write every rule twice. Once in the prompt, so the agent knows what is wanted, and once as code that runs before or after the work and can refuse it. If the second copy does not exist, say out loud that the rule is advice.
It costs	Two places to change when a rule changes, and a test that holds them together.
Here	<code>steps/01-boundary/policy.ts</code> reads the ownership graph and refuses before any agent starts, which is why <code>make prove</code> can break it on purpose and watch it fail.
On a platform	The same graph is described to the agent in <code>AGENTS.md</code> and consulted by nothing. The run recorded in the findings built a refused path and skipped an acceptance step, both while being told not to.

The executor is an interface executor-as-interface

Problem	Every part of the system needs an agent to do work, and binding to one makes the whole thing a wrapper around a product that will change.
Forces	Testing wants determinism and no spend. Teaching wants a real agent. Buying wants the freedom to change the agent later. All three pull against hard-coding one.
The move	Define an executor as any command that takes a prompt on standard input and exits. Everything above it depends on the interface, never on the product.
It costs	One indirection, and usage figures an agent reports rather than ones you measured.
Here	<code>steps/lib/executor.ts</code> , with a replay executor that makes <code>make check</code> free and offline.
On a platform	The agent role on a working phase. Swapping the model is a settings change rather than a code change.

The name is the claim deterministic-claim

Problem	Two scheduled runs wake up for the same item and both start work on it.
Forces	A lock table is correct and is a service to operate, with an availability story and a question about what happens when a holder dies.
The move	Derive the branch name from the item so both runs compute the same one, and push without force. The first push creates the ref and the second is rejected.
It costs	Nothing, unless somebody adds a force flag, which is why there is no flag to pass.
Here	<code>branchFor()</code> in <code>steps/02-execution/worktree.ts</code> , proven by a real push race in the tests.
On a platform	The server owns session identity, so the race does not arise. The idea still applies to anything you schedule yourself.

A missing check is louder than a failing one fail-closed-gate

Problem	A repository with no test command produces a run that looks exactly like a passing one.
Forces	Absence and success are indistinguishable from outside. Treating absence as success is the default in almost every pipeline, because it takes no code.
The move	Make the gate report three states rather than two. Missing is neither a pass nor a failure, and it stops the run.
It costs	A repository that has not finished wiring its checks cannot ship through the factory yet, which is the point.
Here	<code>steps/04-verification/gate.ts</code> returns <code>MISCONFIGURED</code> with exit 2.
On a platform	A tool result rule reading the verdict, which refuses to read a missing check as either outcome.

One line nothing may restate verdict-line

Problem	An agent reports that the checks passed, and that report is the only thing anyone reads.
Forces	An agent summarising is useful everywhere except in the sentence somebody will act on.
The move	End the gate with one exact line. Downstream may quote it and may not paraphrase it, and a quotation that does not match the original is a downgrade.
It costs	Slightly stilted output, and a rule people have to be told once.
Here	<code>VERDICT: PASS every selected check passed</code> , compared exactly in <code>judge()</code> .
On a platform	The same line, carried in the tool result the rule reads.

Tests the writer never saw holdout-sample

Problem	The same run writes the code and the tests, so a green suite means the code agrees with itself.
Forces	Tests written with the change are the most useful ones to have. They are also the least independent.
The move	Keep a small set of behavioural checks out of reach of the implementing agent and run them afterwards.
It costs	A second place to maintain, and it must stay small or it becomes a second suite.
Here	<code>steps/04-verification/holdout/</code> , outside the target and never routed.
On a platform	A checking phase whose role has different tools from the building one.

The writer never grades the work cold-review

Problem	The thing that produced a change is the worst judge of whether it is right.
Forces	The account of how work went is genuinely useful context, and it is also what makes a reviewer confirm rather than check.
The move	Give a second agent the task and the diff, and deliberately not the story of the attempt.
It costs	Minutes per change, and occasional rejections of work that was fine.
Here	<code>judge()</code> in <code>steps/04-verification/verifier.ts</code> , with recorded reviews.
On a platform	The Review board, which gives a pull request its own session rather than continuing the authoring one.

Undo the change and watch the test fail negative-proof

Problem	A test that has never been red is not known to test anything.
Forces	Nobody wants to break working code to check a test. It is also the only check that proves the test is connected to the behaviour.
The move	Revert the change, run the new test, require red, restore. Do it every time rather than when somebody remembers.
It costs	One extra run of one test.
Here	<code>make demo STEP=04 -- negative-proof</code> , which does it for real on a scratch module.
On a platform	Not built in. It is a step you add to the checking role, and the most portable idea here.

Code decides what a verdict may claim evidence-rule

Problem	The result a person reads is whatever the agent decided to say.
Forces	Investigation needs freedom. The sentence somebody acts on needs none at all.
The move	After the agent stops talking, ordinary code reads the artifacts the run left and downgrades any claim whose proof is missing.
It costs	The rules have to be written, and each one needs a reason.
Here	<code>judge()</code> : no reverted proof, no holdout, or a misquoted gate line all downgrade.
On a platform	A tool result handler, which sees what a tool returned rather than what was said about it.

The limit is review attention back-pressure

Problem	Generation is cheap, so the queue of things waiting for a person grows without bound.
Forces	More agents feels like more throughput. The binding constraint is one person understanding a change well enough to stand behind it.
The move	Cap how many items may wait on a person, and refuse to start work when the cap is reached.
It costs	Idle capacity, which is the correct thing to have.
Here	<code>STOP_IF: awaiting_review >= n</code> in the charter, checked before anything else runs.
On a platform	Not built in. It is yours to add, and it is the first thing people delete.

Freedom follows blast radius, not size autonomy-by-consequence

Problem	Deciding what an agent may do unattended by how big the change is.
Forces	Size is easy to measure and is the wrong axis. A ten thousand line migration on an unlaunched project is safer than fifty lines in a payment path.
The move	Grade autonomy by how bad the worst outcome is and how easily it is undone, per area rather than per repository.
It costs	Somebody has to decide, area by area, and write it down.
Here	The <code>autonomy</code> column in <code>.factory/targets.json</code> , with a reason beside each.
On a platform	A transition policy, which can also demand recorded acceptance before code exists.

Some paths are never touched protected-path

Problem	The area where a mistake is most expensive is the area an agent will eventually touch.
Forces	A blanket refusal stops the factory being useful. No refusal at all means the first surprise is the expensive one.
The move	List the paths that are never modified unattended, each with the reason, and have the list protect itself.
It costs	A route for work that legitimately needs those paths, which is a person.
Here	The <code>protected</code> block in the charter, which protects the charter and the graph beside it.
On a platform	The same list, expressed in the policy that allows or rejects a move.

Route, do not pile context-routing

Problem	The instructions file grows every time something goes wrong, and accuracy falls as it grows.
Forces	Each addition works, which is why it keeps happening. The sentence that matters is now competing with forty that do not apply.
The move	Compute what a task receives from what it will touch: the rules for those areas, the skills for that kind of work, and nothing else.
It costs	A router to maintain, and the discipline to keep it small enough not to need a diagram.
Here	<code>steps/03-context/router.ts</code> , which records what it withheld beside what it included.
On a platform	The prompt a skill invocation builds, plus the repository the sandbox checks out.

Hand back the reason, not the rejection doer-and-tester

Problem	A failed check comes back as a status, so the next attempt repeats the mistake.
Forces	One agent grading its own output has already spent its reasoning defending it.
The move	Split the roles. The tester returns what failed, where, and what should have been true, and the next attempt starts from that.
It costs	Two roles to run, and a loop that needs declared endings or it will not have one.
Here	<code>handback()</code> in <code>steps/05-loop/loop.ts</code> , with attempt and time budgets.
On a platform	A retry outcome on the checking phase, routing back to building.

Separate where work is from who decides phase-and-policy

Problem	Process and permission get tangled, so changing one changes the other by accident.
Forces	A board is about where work is. A policy is about who may move it. Mixing them makes every rule a layout change.
The move	Declare phases and the moves between them, then declare separately which moves a person must make.
It costs	Two things to read instead of one, and worth it the first time the process changes.
Here	The graph declares areas and the charter declares who decides, and neither restates the other.
On a platform	<code>defineBoard()</code> for phases, <code>transitionPolicy</code> for who may move a card.

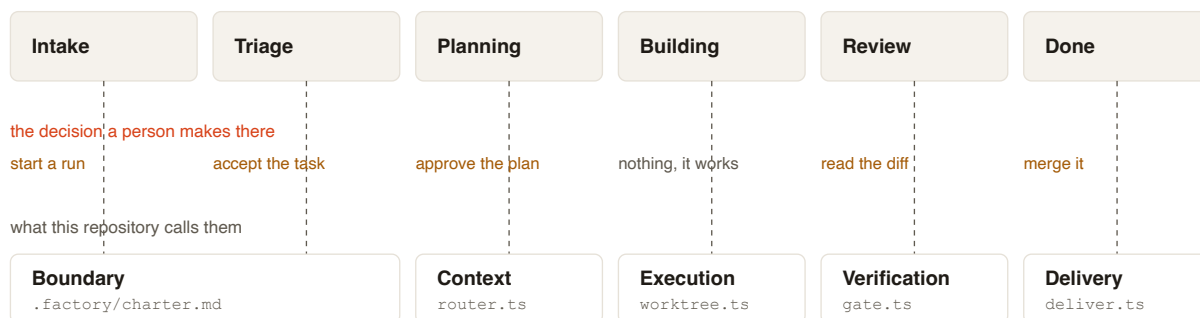
The same
architecture,
bought
instead of
built

On a platform

Phases are the layers, named differently board-phases

A hosted factory gives you a board whose columns are stages, where entering a stage starts an agent in a declared role and leaving one runs an action. Intake, triage, planning, building, review, done. The mapping to what this repository builds by hand is close to one to one, which is the point of building it by hand first.

what a hosted factory calls them



The names differ and the shape does not. Recognising that is what turns the buy decision into an engineering one.

Figure 2. A hosted factory's phases against the layers built by hand. The middle row is the part that never transfers: which decisions stay with a person is yours to state on either substrate.

Recognising the mapping is what makes the buy decision an engineering decision. You are not choosing between a product and nothing. You are choosing who maintains the parts you now understand.

The policy is still yours transition-policy

The part that never transfers is the judgement. A platform supplies the board, the sandboxes and the sessions; what may move to done without a person, which work needs recorded acceptance before any code exists, and where the money path sits are still yours to state. On a platform they are a function that allows or rejects a move.

So the last step here is not a migration. It is the same charter expressed against a different substrate, which is the honest test of whether the architecture was real.

Run sheets for each session sit in `teach/`. The build is scored by `make score`, and the figures on these pages are read from the recorded run by a test.

Part two

The sessions, and what each one is responsible for

Each sheet below is one live session. It carries the timing, the talk track and the commands, and it cites the ideas in part one rather than explaining them a second time.

Lightning lesson, sixty minutes

Design a Software Factory for Your Coding Agents

Architecture session. No live coding. The room leaves with a map they can hold against their own repository, and one named gap.

Twenty minutes ahead, alone

Before anyone joins

Goal The demo is warm and the console has data in it. Nothing here happens while people watch.

Say Nothing. This segment is setup.

Run

```
$ make check
$ make score
$ make demo STEP=06
```

- Check**
- The score prints and every phase is green.
 - The console is serving and lists finished runs with their timings.
 - Your editor is on a clean branch, because the harness will create worktrees and you want that visible.
-

Ask *Nobody. Confirm the room can read a terminal at the back before you start.*

0:00 to 0:08

The afternoon that disappears

Goal	Everyone recognises the problem as theirs before any architecture appears.
Say	• Open with the shape of the day, not the technology. The function took seconds. Then came re-explaining the codebase, checking what else moved, and tidying the pull request. • Name the wrong conclusion out loud: that the model needs to improve. Ask who has responded to a bad result by adding another paragraph to their instructions. • Land the reframe. The generator is not the missing piece. There is nowhere safe for it to run, no reliable way to give it what it needs, and nothing checking its work before it reaches you.
Run	Nothing yet. Resist opening a terminal here.
Room sees	Slides or a whiteboard only. Faces, not a screen.
Ask	<i>How long is your instructions file now, and when did you last read all of it?</i>

0:08 to 0:28

The map

Goal	Attendees can name all six jobs and say which failure each one removes.
-------------	---

Say	• Walk the six in order, giving each one a single sentence and a symptom. Do not define them abstractly; define them by what goes wrong when they are missing. • Spend the longest on the second and fifth, because those are where the room's pain actually is. • Say the sentence that reframes the rest of the hour: these fail separately, so the fix differs depending on which one is absent.
------------	---

Run	Show the map in the concept spine and scroll through it.
------------	--

Room sees	• The six-layer diagram, one heading at a time. • The symptom under each, said aloud rather than written on the slide.
------------------	--

Ask	<i>Pick the symptom you had this week. Which job was missing?</i>
------------	---

0:28 to 0:40

Routing beats piling

Goal The room understands why their instructions file stopped helping, and what replaces it.

- Say**
- Describe the growth curve honestly. Each addition works, which is why it keeps happening, and accuracy still falls.
 - Explain the competition for attention: the sentence that matters is now one of forty, thirty-nine of which are irrelevant to this task.
 - Show the alternative as a calculation rather than a philosophy. Files to be touched, owners of those files, rules for those owners, skills for that kind of work.
-

Run

```
$ make demo STEP=03
```

- Room sees**
- Two tasks, side by side, receiving visibly different material.
 - The frontend task never being shown the money rules, and the reason why printed next to it.
-

Ask *What does a new engineer get told on day one that your agent is never told?*

0:40 to 0:52

The whole loop, once

Goal	They see the finished thing move, so the layers stop being a diagram.
-------------	---

Say	• Narrate the stages as they happen and keep naming which job is doing the work. • Stop on the refusal and give it a beat. A run declining to touch the money path is the most important thing on the screen, and it looks like nothing. • Close the loop at the pull request and say plainly that this is where the machine stops.
------------	---

Run

```
$ make demo STEP=06
```

Room sees	• An item claimed, a worktree created, checks running, a verdict, a draft pull request. • A second item stopping with a policy reason instead of a change. • The console updating with the stage timings for the run that just finished.
------------------	--

Ask	<i>Which of those stages would you have let run unattended on your repository this morning?</i>
------------	---

0:52 to 1:00

One gap each

Goal	Nobody leaves with a reading list. Everybody leaves with one named absence.
-------------	---

Say	• Ask them to write down which of the six they do not have, in their own words, before the call ends. • Say where the next sessions go and what each one builds, briefly, without selling.
------------	--

Run	Leave the console on screen while taking questions.
------------	---

Room sees	The repository address, and the concept spine open at the overview.
------------------	---

Ask	<i>Name the one you are missing. Type it in the chat so the room sees the spread.</i>
------------	---

0:35 to 0:45

The same policy, read by a product

Goal They have seen the charter they just wrote being obeyed by software none of us built.

- Say**
- Point out that the board is reading the same `.factory`` directory the hand-built loop reads.
 - The value of a written policy is that a second implementation can honour it without being told anything else.
 - Name what is missing on purpose: nothing has run yet, so nothing on screen is a claim about quality.

Run

```
$ make factory-up
$ make lab-reset
```

The screenshot shows the Mastra Factory interface. On the left is a sidebar with navigation links: Overview, Supervisor, Work (selected), Review, Activity, Audit log, Work Sessions, Review Sessions, Needs attention (62), Workshop, and Settings. The main area displays a list of issues, each with a title, a status icon, and an 'Investigate' button. The issues are: #644: Keep runs for longer than the console can show, #642: Warn earlier when spend is heading for the limit, #643: Review the pull request that adds a saved view, #641: Record how many tool calls a whole run made, and #640: Show what a thousand tokens costs on each r... Above the issues is a workflow diagram with stages: Intake, Triage, Planning, Building, Review, Done, and Cancelled. At the top right, there are toggle switches for 'Auto-start runs' and 'Auto-approve plans', both of which are currently off.

Every issue the repository declares arrives from one place. Nothing has started, and both switches are off.

Show on the board



The charter is not read by magic

Mastra Factory has no knowledge of a `.factory` directory. What it reads is `AGENTS.md`, and the only reason these rules bind anything is that the file points at them and repeats the protected list. Delete it and every limit here becomes a comment.

Both switches are off, deliberately

Arrival and action are separate. An issue landing on Intake is not a run starting, and the gap between those two is where a person decides whether this is work at all.

Six cards, one repository

Nothing on this board knows what the ledger is. It knows a repository was connected and that its issues arrive here.

Room does

- Watches six cards appear without anybody filing them.
- Notices that Auto-start runs is off, so arrival and action are separate things.

Ask

If your policy file vanished, which of these six would a stranger get wrong first?

Concepts for this session: six layers, boundary, context, skills, execution, verification, delivery, the growing prompt.

Lightning lesson, sixty minutes

Build Your First Software Factory Execution Harness

Hands on. Attendees follow along in their own repository and finish with an isolated place for agent work.

Twenty minutes ahead

Setup you do alone

Goal Worktrees can be created and removed instantly, with nothing left over from an earlier rehearsal.

Say Nothing. Preparation only.

Run

```
$ git worktree list
$ make clean
$ make demo STEP=02
```

Check

- Only your main checkout is listed before you begin.
- A rehearsal leaves nothing behind, so the count is the same afterwards.

Ask *Nobody. Widen the terminal font now, because this session is watched closely.*

0:00 to 0:10

Why your checkout is the wrong place

Goal	The room stops seeing isolation as ceremony and starts seeing it as the fix for something that already bit them.
-------------	--

- | | |
|------------|--|
| Say | <ul style="list-style-type: none">• Collect the war stories first. Ask who has had an agent commit something half-finished, or write into the branch they had just switched to.• Reframe each story as a missing place rather than a misbehaving tool.• State what you are about to build and what it will cost: about a second per task, and a directory you can look inside when something goes wrong. |
|------------|--|
-

Run	Terminal open, nothing typed.
------------	-------------------------------

Room sees	An empty prompt and their own stories on the whiteboard.
------------------	--

Ask	<i>Who has lost work to a background run? What did it cost to recover?</i>
------------	--

0:10 to 0:28

A second checkout, built live

Goal Everyone has created one, written a file in it, and confirmed their own branch never moved.

- Say**
- Explain the object store once: one repository, several checkouts, no duplicated history and no second clone to keep current.
 - Build it in front of them rather than opening the finished file. Create, list, write, remove.
 - Call out the removal explicitly. A design that is hard to undo does not get adopted, and this one is a single command.
-

Run

```
$ node steps/02-execution/demo.ts create 142
$ git worktree list
$ git status
$ node steps/02-execution/demo.ts remove 142
```

The implementation is `steps/02-execution/worktree.ts`, and it is short enough to read on screen.

- Room sees**
- Two entries in the worktree list, then one again.
 - A clean status in the main checkout throughout, which is the whole point.
-

Ask *Your build needs installed dependencies. Where do those live now, and what does that cost per task?*

0:28 to 0:42

Two runs, one item

Goal They understand how ownership is settled without anything new to operate.

- Say**
- Pose the collision before showing the answer. Two scheduled runs wake up, both see the same item, both start.
 - Let someone propose a lock, then price it honestly: a service, its availability, and what happens when a holder dies.
 - Reveal the alternative and let the surprise land. The name is derived from the item, so the loser finds out by being rejected.
-

Run

```
$ node steps/02-execution/demo.ts race 142
```

- Room sees**
- Two runs starting together on the same item.
 - One continuing, one stopping with the rejected push printed underneath.
-

Ask *What else in your pipeline would be simpler if the coordination lived in a tool you already run?*

0:42 to 0:54

What it can reach from in there

Goal Attendees can state limits for one task in their own repository and know what happens when one is exceeded.

- Say**
- Separate the two questions cleanly. Isolation answered where. Limits answer what it can touch once it is there.
 - Argue for declaring limits per task rather than globally, using the lockfile example: that task genuinely needs the network and most do not.
 - Insist on the ending. Exceeding a limit stops the run. A warning in a log is not a limit.

Run

```
$ node steps/02-execution/demo.ts limits
```

Declared in `steps/02-execution/limits.ts`, where a reviewer can read them.

Room sees

- A permitted write succeeding.
- A write outside the allowed set refused, naming the path and the rule.
- A command not on the list refused before it runs.

Ask *List three commands you would never want a background run to execute. Why those three?*

0:54 to 1:00

Take it home

Goal	Everyone has the harness working against their own repository before they close the call.
-------------	---

Say	• Give the one instruction to try tonight, and say it slowly enough to be written down. • Warn about the dependency cost up front so it is expected rather than discovered.
------------	---

Run	Leave the worktree list and a clean status side by side on screen.
------------	--

Room sees	The repository address and the two files worth reading first.
------------------	---

Ask	<i>Any repository where a second checkout would not work? Bring it to the next session.</i>
------------	---

0:35 to 0:45

Somebody else's sandbox, same rule

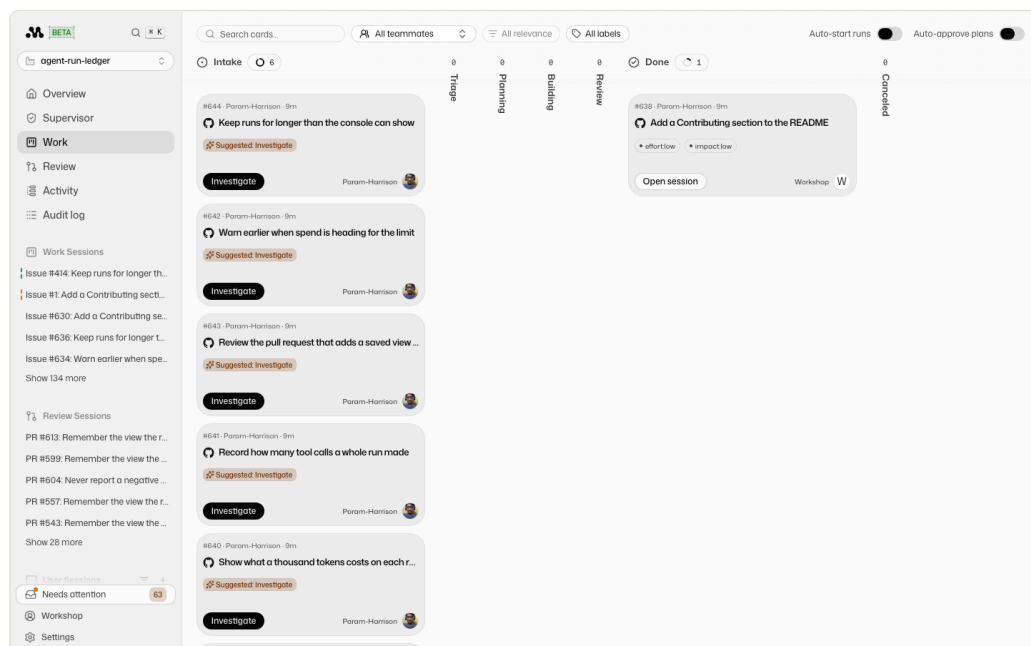
Goal They can say why an isolated checkout is a property of the harness rather than a courtesy.

Say

- Two harnesses now: the one built in this lesson and the one inside the product.
- Both give a run its own tree, and for the same reason, which is that a shared tree makes two runs indistinguishable when one of them breaks.
- Show where the checkout lives, and that deleting it costs nothing.

Run

```
$ ls ../mastra/.sandboxes || true
```



A run holds a checkout of its own. The repository on your disk is untouched while it works.

Show on the board

Work to a card in Building to its session to the file tree

A checkout of its own, every time

The session holds a clone under `.sandboxes`. Delete it while the run is going and the run dies; delete your own working tree and nothing happens, which is the whole point of the boundary.

The one that cost me an evening

A review once checked a pull request out in the Factory server's own directory rather than a sandbox. `docker-compose.yml` and `package.json` vanished and the only symptom was "compose file is invalid". Ask the room where they would have looked first.

Sandboxes are cattle

They are named by run, not by branch, and nothing in them is worth keeping once the pull request exists.

Room does

- Opens the sandbox directory and confirms it is a real clone.
 - Confirms their own working tree has no new branch.
-

Ask

What breaks first if two runs share one checkout: the build, the test, or the diff?

Concepts: worktree per task, the branch as claim, limits, executor interface.

Lightning lesson, sixty minutes

Enforce Code Quality with Automated Agent Verifiers

The session that changes how people review. Most of the hour is spent on one idea: undo the change and watch the test go red.

Fifteen minutes ahead

Arm the demonstration

Goal	A recorded run is ready that produced a genuinely flawed change, so the rejection is real rather than staged.
Say	Nothing. Loading a recording.
Run	<pre>\$ make demo STEP=04</pre>
Check	• The gate prints its result line and nothing summarises it afterwards. • The rejection you intend to show is in the recording, with its reason.
Ask	<i>Nobody. Have the flawed diff open in a second window, ready to show.</i>

0:00 to 0:10

Code that reads correctly

Goal	The room agrees that reviewing plausible work is more expensive than reviewing obviously broken work.
Say	• Put the flawed diff on screen with no commentary and give them thirty seconds to find the problem. • Most will not, and that is the lesson. Reveal it, then point out that the tests were green. • Draw the conclusion carefully: the problem is not that the work was bad, it is that nothing independent had looked before you did.
Run	The diff on screen. No commands.
Room sees	A change that looks fine, with a passing suite beside it.
Ask	<i>How much of your review time goes on work that turns out to be correct?</i>

0:10 to 0:24

One line nobody may rephrase

Goal Attendees can state why a missing check must be louder than a failing one.

- Say**
- Show the script ending in a single result line and explain the rule around it: quoted exactly, never summarised.
 - Delete the test command in front of them and run it again. The result is not green and it is not a failure either.
 - Make the argument explicit. Absence and success look identical from the outside, and only one of them is safe to act on.
-

Run

```
$ bash steps/04-verification/gates.sh
$ bash steps/04-verification/gates.sh --without-tests
```

The script is `steps/04-verification/gates.sh`.

- Room sees**
- A passing result line, then a misconfigured one with a different exit code.
 - No prose anywhere claiming the run was fine.
-

Ask *If your pipeline lost its test step tonight, how long before anyone noticed?*

0:24 to 0:42

Undo the fix

Goal Everyone leaves able to apply this to code that has no agents anywhere near it.

- Say**
- State the check in one sentence and then do it, rather than explaining it twice.
 - Show the good case, then plant a test that passes whatever the code does and show it surviving. That contrast is the teaching moment.
 - Say the general version out loud, because it is the most portable idea in the course: a test that has never failed is not known to test anything.
-

Run

```
$ node steps/04-verification/demo.ts negative-proof
$ node steps/04-verification/demo.ts negative-proof --toothless
```

- Room sees**
- The change reverted, the new test going red, the change restored.
 - The second run reporting that the test stayed green while its subject was broken.
-

Ask *Take the last test you wrote. Would it have gone red without your change?*

0:42 to 0:54

Reading the diff cold

Goal They understand why the reviewer is deliberately denied the story of the work.

Say

- Describe the anchoring problem from human review, which everyone recognises, then apply it to a second agent.
- Show what the reviewer receives and, more importantly, what it does not receive.
- Return to the flawed diff from the opening and let the cold reviewer find it. That closes the hour on evidence rather than assertion.

Run

```
$ node steps/04-verification/demo.ts verify
```

Room sees

- The reviewer's input: the task and the diff, with no account of the attempt.
- A rejection naming the behaviour the change added that the task never asked for.

Ask *What does your reviewer currently read first, and does it change what they conclude?*

0:54 to 1:00

Cost and close

Goal Nobody leaves thinking this is free.

Say

- Be honest about the price. An independent pass adds minutes and sometimes rejects work that was fine.
- Give the mitigation rather than the excuse: bound the stage, and let it consume evidence the implementation already produced instead of repeating it.

Run Show the stage timings from the recorded run.

Room sees Where the minutes actually went, with checking larger than writing.

Ask *What would you accept paying, in minutes, for an independent read of every change?*

0:35 to 0:45

A reviewer that can say no

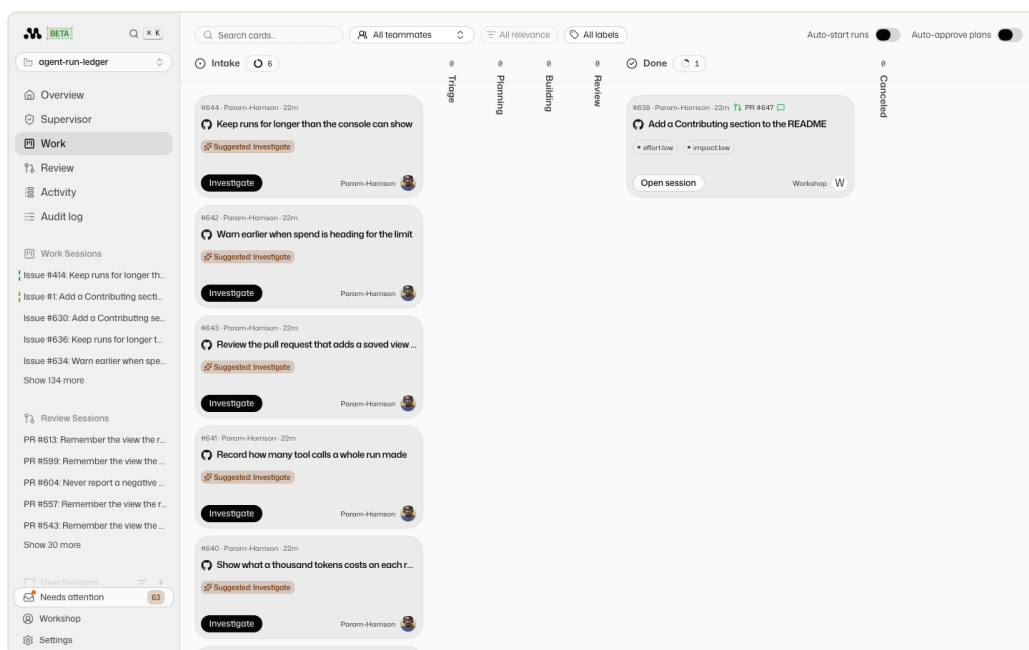
Goal They accept that a reviewer which never rejects anything has not been tested.

Say

- The interesting artefact is the refusal, not the approval.
- Read the two findings aloud and note that one of them is a weakened assertion, which no path rule could have caught.
- This is why the charter forbids editing a test to make a suite green: it is a property of the change, not of a filename.

Run

```
$ make lab-reset
```



The verdict line, with the two findings that produced it.

Show on the board

Review to the pull request to Conversation to the verdict line

The verdict is the first line, on purpose

A reviewer that buries its conclusion in prose cannot be checked by anything.

One line, one of two values, and everything after it is the argument for it.

A weakened assertion is not a path rule

Changing toBeCloseTo(1, 10) to toBeGreaterThanOrEqual(0) touches no protected file. No glob catches it, which is why the charter forbids it in words and the reviewer has to be the thing that notices.

A picture is the one artefact a test cannot read

Four screenshots in this repository were a loading spinner, and every assertion around them passed. Show the room the guard that refuses to photograph a page still saying it is loading.

Room does

- Reads the verdict line and both findings.
- Identifies which finding a glob could have prevented and which it could not.

Ask

Your suite went green after an edit to a test. What would tell you which kind of edit it was?

Concepts: the verdict line, holdout tests, the cold reviewer, the negative proof, Doer and Tester.

Lightning lesson, sixty minutes

Automate Pull Requests with Background Coding Agents

A live run from an untouched issue to a reviewable pull request. The demonstration is the session; commentary fits around it.

Thirty minutes ahead

Reset the board

Goal The scratch repository is back to its seed, with the three prepared items open and none of them touched.

Say Nothing. This is the riskiest preparation in the series, so leave time for it.

Run

```
$ make lab-reset
$ make demo STEP=06 -- --dry-run
```

Check

- Three open items and no pull requests.
- The dry run plans all three and writes nothing.
- A second terminal ready, because the run and the console are watched together.

Ask *Nobody. Decide now what you will do if the network drops, and keep the recording as the fallback.*

0:00 to 0:12

An issue an agent can act on

Goal Attendees can tell the difference between an item that is ready and one that only looks ready.

- Say**
- Read the three prepared items aloud and ask the room to sort them before the machine does.
 - Draw out what makes the difference: a checkable finishing condition, the files it is expected to touch, and whether the intent needs a person.
 - Point out that triage is a separate stage for exactly this reason. Merging it into implementation is how a narrow request becomes a wide change.
-

Run

```
$ node steps/06-delivery/demo.ts triage
```

- Room sees**
- Each item labelled, with the reason recorded beside it.
 - One sent to a person, and the question it needs answered.
-

Ask *Open your own backlog. How many of the top five could a machine start on today?*

0:12 to 0:36

Let it work

Goal The room watches a change get made and checked without anyone steering it.

- Say**
- Start it, then stop talking for a moment so the room registers that nobody is driving.
 - Narrate each stage as it appears and keep naming which layer is responsible.
 - Use the waiting time for the question the room always asks, which is what happens when it gets something wrong. The answer arrives in the next segment.

Run

```
$ make demo STEP=06
```

-
- Room sees**
- The item claimed and a worktree appearing.
 - Context assembled, then a change, then three suites running in three languages.
 - Both verdicts, then a draft pull request opening.
-

Ask *While this runs: what would you need to see attached before you would read one of these?*

0:36 to 0:46

The one it will not do

Goal The room sees a refusal treated as a successful outcome rather than an error.

- Say**
- Set it up before running it. This item needs the pricing rules, and pricing decides what a customer pays.
 - Run it and let the stop sit on screen without narration for a second.
 - Explain what would have happened without the rule: a plausible change to money, verified by tests the same run wrote.
-

Run

```
$ node steps/06-delivery/demo.ts refused
```

- Room sees**
- The run stopping before any file is written.
 - The rule that stopped it, quoted from the charter, and the item routed to a person.
-

Ask *Which directory in your repository would you put on that list first?*

0:46 to 0:56

What arrives for a person

Goal They see that the decision is unchanged and only the preparation for it is different.

Say

- Open the pull request and read its evidence section top to bottom without editorialising.
- Say the sentence the room needs to hear: the reviewer is not removed, they arrive after everything a machine can check is done.
- Show the console and let the stage timings make the point about where time goes.

Run

```
$ node steps/06-delivery/demo.ts open-pr
```

Room sees

- A draft pull request with both verdicts, the negative proof and the cost attached.
- The console listing the run, with its stages as a waterfall.

Ask *Is that enough to review in five minutes? What is missing for you?*

0:56 to 1:00

Where the ceiling is

Goal Nobody leaves believing more agents is the next move.

Say

- Name the real limit: how many decisions can be waiting on one person, not how much can be generated.
- Mention that several verified requests on one base branch still have to be integrated by somebody.

Run Leave the pull request list on screen.

Room sees More than one request awaiting review, and the production stopping.

Ask *How many pending decisions can you genuinely absorb in a week?*

0:35 to 0:45

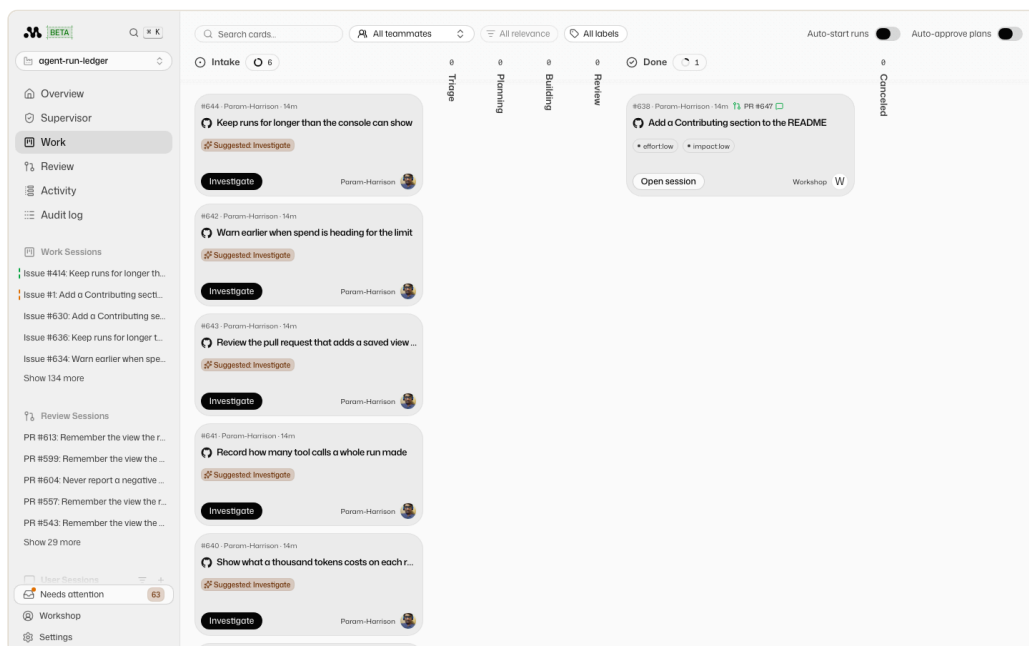
The pull request as the interface

Goal They can describe the hand-off point between a machine and a person in one sentence.

- Say**
- Every route in this material ends at a draft, on every tier, without exception.
 - The pull request is where the evidence lives, which is what makes it reviewable by someone who watched none of the run.
 - Contrast that with a chat transcript, which nobody can audit six weeks later.

Run

```
$ gh pr list --repo learnwithparam/agent-run-ledger
```



The change, its evidence, and nothing merged.

Show on the board

Review to the pull request to Files changed to the plan artefact

The evidence travels with the change

The plan the run wrote is committed beside the diff, so a reviewer who watched none of it can see what was intended before reading what was done.

Nothing merges itself, on any tier

Every route in this material ends at an open pull request. The charter says so and the four shapes do not argue about it.

A decision is not the work

The board records a decision succeeding within seconds while the agent goes on working for minutes. Anyone timing a stage from the decision record will report a factory four times faster than it is.

Room does

- Opens the pull request and finds the commands the run actually executed.
- Confirms nothing was merged by anything except a person.

Ask

What in this pull request would you need if you were reviewing it a month from now?

Concepts: the queue, a request with its evidence, the merge gate, the run record.

Design Your Software Factory Architecture

Everyone arrives with a repository they are allowed to change. They leave with a written policy for it that a run can read, and that they could defend in a design review.

0:00 to 0:20

Their repository, not mine

Goal Each person has named the area of their codebase where an error would be most expensive.

Say

- Set the rule for the whole cohort in the first minute: every exercise happens on their code, and the lab exists only to show a worked example.
- Ask them to write down the single directory they would least like a machine to edit, and why it earns that.
- Collect three answers aloud. The reasons differ, and hearing that is what stops the policy becoming a template they copy.

Run

```
$ git clone <their repository> && cd <repository>
$ ls
```

Room does

- Opens their own project beside the lab.
- Writes one directory name and one sentence of justification in the shared document.

Ask *Would a small edit there be safer than a large one somewhere else? Say why out loud.*

0:20 to 0:45

Grading freedom by consequence

Goal They can defend a tier for their repository using recoverability rather than size.

- Say**
- Offer the two candidate axes and let the cohort argue for a few minutes before you settle it.
 - Use the comparison that ends the argument: a huge rewrite on a project nobody uses against a tiny edit to a payment path.
 - Then complicate it usefully. One tier for a whole repository is too blunt once areas differ, which is why the next exercise is per area.
-

Run

```
$ cat $REPO/.factory/charter.md
```

The rules live in the repository they govern. The code that reads them is `steps/01-boundary/policy.ts`, and it names no project.

- Room does**
- Drafts a tier line for their own repository.
 - Argues one case with a neighbour, which surfaces the disagreements early.
-

Ask *What is the worst thing an unattended run could do to you, and how long would undoing it take?*

0:45 to 1:20

Who owns which paths

Goal Everyone has a graph where every directory has exactly one owner, and the build refuses when one does not.

- Say**
- Introduce the graph as the thing that makes policy specific rather than vague. Areas, the paths they own, and the commands that check them.
 - Build the lab's version live, then break it by adding an unclaimed directory so the room sees the refusal.
 - Insist on the property that matters. Guessing an owner is worse than failing, because a wrong guess is silent.
-

Run

```
$ node steps/01-boundary/demo.ts graph
$ mkdir -p target/services/orphan && touch target/services/orphan/main.go
$ node steps/01-boundary/demo.ts graph
```

- Room does**
- Sees every area listed with its paths and its checks.
 - Sees the second run refuse and name the directory nobody claimed.
 - Writes the first version of their own graph.
-

Ask *Which directory in your project was hardest to assign, and what does that tell you?*

1:20 to 1:45

The number that stops production

Goal Each person has committed to a limit on pending decisions, chosen from their own week.

- Say**
- Ask how many changes they genuinely absorb in a week, then ask them to halve it for anything they did not write.
 - Explain what the limit protects: not the machine, the person. When it is reached, starting more work is the wrong move.
 - Show the refusal in the lab so the number is visibly load bearing rather than advisory.
-

Run

```
$ node steps/01-boundary/demo.ts capacity
```

- Room does**
- Watches a run decline to start because the review queue is full.
 - Puts a number in their own charter and says it aloud.
-

Ask *What happens today when that number is exceeded, and who notices first?*

1:45 to 2:00

Before Wednesday

Goal Everyone has a committed policy file in their own repository before the next session.

- Say**
- Set the deliverable precisely: committed, not drafted, because the next session reads it.
 - Warn about the common mistake, which is writing something so cautious that nothing can ever run.
-

Run

```
$ node steps/01-boundary/demo.ts lint-charter <path to theirs>
```

Room does Runs the check against their own draft and fixes what it names.

Ask *Bring the line you are least sure about to Friday.*

1:35 to 1:50

Say

- A limit that has never stopped anything is a sentence, not a limit.
- Walk the evidence backwards: no branch on the remote, no pull request, and a written reason that names the path.
- Then the uncomfortable part. Ask whether the refusal came from the charter or from the wording of the issue, because only one of those generalises.

```
$ make profile NAME=enterprise
$ make lab-reset
```



Work to the money item to Planning to its session

A run that stops because the issue said "a person decides this one" looks identical to one that stops because the charter protects the path. Only the second generalises to their repository, and the difference is a sentence in the record.

Check the remote for a branch, then for a pull request. A refusal that leaves either behind is not a refusal, and this is the check to run on their own factory.

Protected and refuse are different stops

A refused target stops after planning with a plan nobody may act on. A protected path stops before planning starts. Enterprise uses both and scaleup uses one.

Room does

- Applies a shape to their own repository and names one path it would refuse.
 - Checks the remote afterwards and confirms nothing was created.
-

Ask

Which of your protected paths has ever actually stopped a change? If none, is it protecting anything?

Concepts: the charter, autonomy by consequence, protected paths, back-pressure, the ownership graph.

Isolate Agent Work with an Execution Harness

The build session. By the end an agent can change code in their repository while their own branch stays exactly as they left it.

0:00 to 0:15

Reading each other's policies

Goal	The homework gets read by someone other than its author, which is where the vague lines get found.
-------------	--

Say	• Pair them and give a single instruction: find one line you could not act on without asking the author a question. • Bring two examples back to the whole group and rewrite them live. • Make the standard explicit. A policy is finished when a stranger could apply it the same way you would.
------------	---

Run	Nothing. Documents on screen, in pairs.
------------	---

Room does	Edits at least one line based on somebody else's confusion.
------------------	---

Ask	<i>Which line did your partner read differently from how you meant it?</i>
------------	--

0:15 to 1:00

Building it together

Goal Everyone has a working create and remove cycle in their own project, written by them.

- Say**
- Write it in front of them from an empty file rather than opening the finished version. The cohort types along.
 - Stop at each decision and say what the alternative was: a clone instead of a worktree, a random name instead of a derived one, a global cleanup instead of a per-task one.
 - Have everyone prove the same property on their own repository before moving on, because a harness nobody has proven is a harness nobody trusts.
-

Run

```
$ node steps/02-execution/demo.ts create 7
$ git -C .worktrees/fq-7 status
$ git status
$ node steps/02-execution/demo.ts remove 7
```

- Room does**
- Creates a workspace, writes into it, and confirms the main checkout is untouched.
 - Removes it and confirms nothing is left behind.
-

Ask *How long did the first one take on your project, and what dominated that time?*

1:00 to 1:35

Reach, declared per task

Goal Each person has limits their own reviewer could read, and has watched one stop a run.

- Say**
- Insist on the placement before the content. Limits go where a reviewer will see them, not inside instructions to the agent.
 - Work through a task that legitimately needs more reach, so the rule is not simply deny everything.
 - Show the enforcement path and be honest about where it is advisory and where it is real.
-

Run

```
$ node steps/02-execution/demo.ts limits
$ node steps/02-execution/demo.ts limits --violate
```

- Room does**
- Watches a refused write name the path and the rule behind it.
 - Writes limits for one real task in their own project.
-

Ask *Which of your tasks genuinely needs the network, and how would you keep that from becoming the default?*

1:35 to 2:00

This week's build

Goal Everyone knows exactly what to produce and what will be checked on Monday.

- Say**
- State the deliverable as something observable: a task that runs start to finish in isolation on their repository.
 - Predict the two failures they will hit, so neither costs an evening: dependencies not installed in the new workspace, and generated files ignored by the clone.
-

Run

```
$ node steps/02-execution/demo.ts selftest
```

Room does Runs the self-check against their own harness and reads what it reports.

Ask *Post your isolation time in the channel when you have it running. We compare on Monday.*

1:35 to 1:50

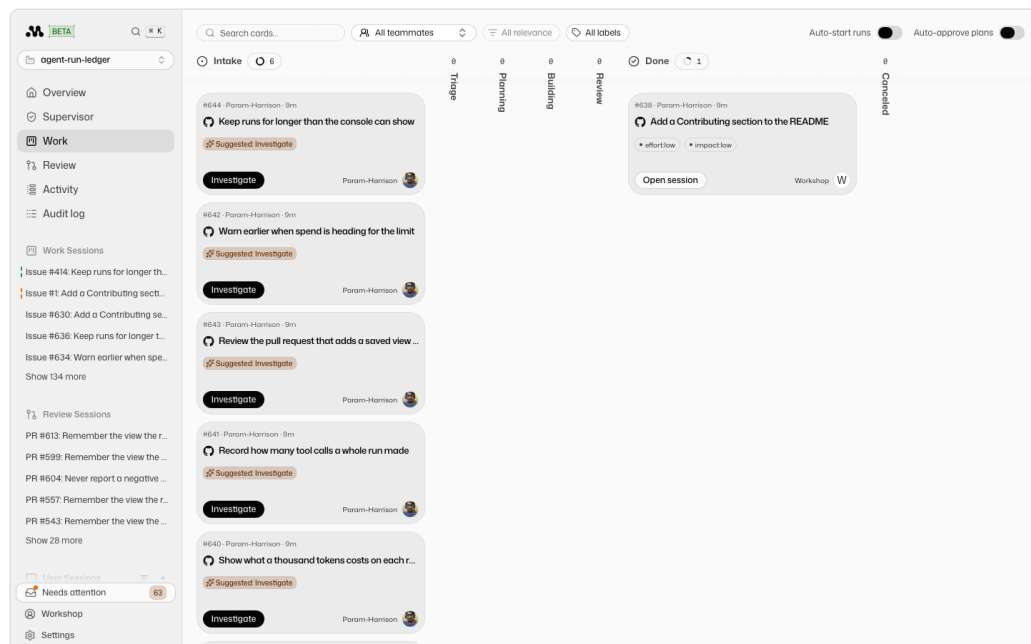
Many runs, one board

Goal They can explain why a queue limit is about attention rather than machine capacity.

- Say**
- Generation is cheap and review is not, so the constraint worth declaring is how many things may be waiting on a person.
 - Show the cap in the charter beside the board and let them find the mismatch if there is one.
 - A factory producing faster than a team can read is not more productive, it is further behind.

Run

```
$ cat $REPO/.factory/charter.md
```



Several items in flight at once, each in a different phase.

Show on the board

Work to the board header to the phase counts to Settings

Work items come from events, never from polling

The reconcile sweep patches and closes items that exist; its counts have no created. The board fills because a webhook arrived, which is why a lab GitHub cannot reach delivers those events to itself.

The cap is about reading, not generating

Compare the number waiting with the stop condition in the charter. Generation is cheap; the constraint worth declaring is how many things may be queued against one person's attention.

Two boards, not one

Issues live on Work and pull requests live on Review, with their own phases.
Teams that conflate them report a week of finished work that nobody has read.

Room does

- Finds the stop condition in the charter.
- Counts what is waiting on the board and compares the two numbers.

Ask

At what number of open pull requests does your team stop reading them properly?

Concepts: the worktree, claiming by branch, declared limits, the executor.

Route the Right Context to Every Task

The session where the instructions file gets taken apart. Everyone leaves with a router they can test, and two skills lifted out of prompts they were rewriting by hand.

0:00 to 0:20

An autopsy on your instructions

Goal Everyone has sorted their own instructions into three piles and seen how little of it any one task needs.

- Say**
- Give them the three piles and let them do the sorting themselves: facts about this repository, ways of working, and things that apply to one task only.
 - Ask for a show of hands on how much landed in the third pile. It is usually most of it, and seeing that is the argument.
 - Name what each pile becomes before moving on, so the rest of the session has somewhere to put things.
-

Run

```
$ wc -l <their instructions file>
```

Room does Annotates their own file with three markers and reports the split.

Ask *Which line in there was added after one bad result and never revisited?*

0:20 to 1:05

Assembling context as a calculation

Goal Each person has a function that turns a task into its material, and a test that pins what it returned.

Say

- Derive the inputs with the room rather than presenting them. What does the task say it will touch, who owns those paths, what rules apply there.
- Build the lab's router live and keep it small. If it needs a diagram it has become a framework.
- Write the test immediately after, because the point of code over prose is that it can be asserted. Give the router a task and pin what came back.

Run

```
$ node steps/03-context/demo.ts route 12
$ node steps/03-context/demo.ts route 31
```

The router is `steps/03-context/router.ts`.

Room does

- Sees two tasks receive visibly different material, with the reason for each inclusion.
- Writes a first router against their own ownership graph.

Ask *What did your router include that the task did not need? Remove it and say what broke.*

1:05 to 1:40

Lifting a procedure out of a prompt

Goal Everyone has written one skill that states an outcome and its evidence, without scripting the keystrokes.

- Say**
- Ask what they have retyped more than three times this month. That is the first skill, and they already know what it is.
 - Show the failure mode before the good version: a long procedure that is worse than the agent's own judgement and goes stale within a month.
 - Give the three parts and hold them to it. The outcome, the constraints that are not obvious from the code, and what proves it is done.
-

Run

```
$ ls steps/03-context/skills
$ cat steps/03-context/skills/change-the-shared-contract.md
```

- Room does**
- Reads one worked example, then closes it and writes their own.
 - Swaps with a neighbour to check the outcome is stated clearly enough to act on.
-

Ask *Could this skill be copied to a different repository unchanged? If not, some of it is context.*

1:40 to 2:00

The shared definition trap

Goal They understand why a naive mapping of files to areas ships a break, and have the dependency edge that fixes it.

- Say**
- Pose it as a puzzle. One file changed, one area owns it, one suite ran, everything green, production broken. What happened?
 - Draw out the edge: three areas are built against that definition, so all three are affected.
 - Note the second use of the same graph. It also tells the router which rules a task needs, so this is one calculation serving two purposes.
-

Run

```
$ node target/tools/affected.mjs --since HEAD~1
```

Room does Sees one edited definition select three areas and three different check commands.

Ask *Where is your shared definition, and how many places restate it today?*

1:35 to 1:50

What it read before it decided

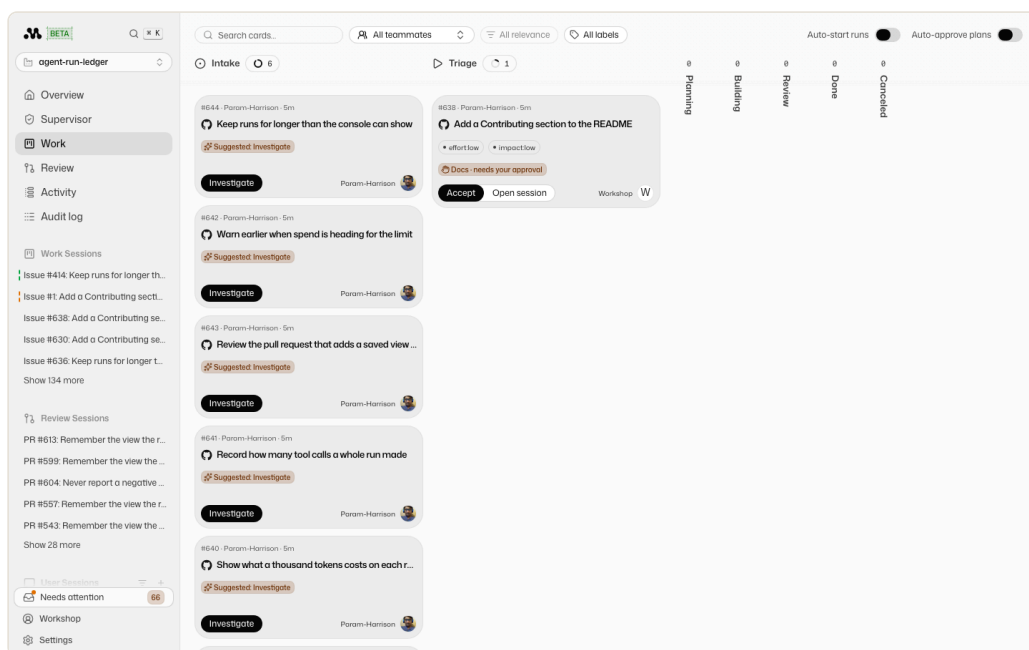
Goal They can tell an assessment grounded in the repository from one grounded in the issue text.

Say

- Read the file paths in the assessment and check every one exists.
- An assessment that names line numbers has opened the file; one that paraphrases the issue has not.
- This is the cheapest quality signal available before any code is written, and it costs one read.

Run

```
$ make status
```



The assessment, with the files it names and the confidence it claims.

Show on the board

Work to a triaged card to its session to the assessment table

Every path it names, check one

An assessment that cites a file and a line number has opened the file. One that restates the issue in different words has not, and the two read alike until somebody checks.

The best outcome is sometimes no change

One run here answered a request for a cost column by proving the column already existed, with line numbers and five real currency values read from a live service. The issue was wrong and the run was right.

Confidence is a claim, not a feeling

The table states high or low confidence. Ask what evidence would have to be missing for that word to be wrong, and whether the run went and looked for it.

Room does

- Verifies two cited paths against the repository.
- Marks the assessment as grounded or not, and says which evidence decided it.

Ask

Which sentence in this assessment could have been written without opening the repository?

Concepts: the router, writing a skill, why prompts grow, affected areas.

Make Agents Prove Their Work

The longest build of the course. Everyone finishes with checks that fail closed and a reviewer that reads their diffs without being told the story.

0:00 to 0:30

One script, one result line

Goal Everyone's checks are behind one command that says one thing, and a missing check is louder than a failing one.

Say

- Start from where they are. Most have commands scattered across a pipeline file, a readme and somebody's shell history.
- Gather them into one script live, then add the ending that makes it useful: a single line, exactly worded, that nothing may restate.
- Do the deletion demonstration on their own project, not the lab. Remove a required command and show that the result is neither green nor a failure.

Run

```
$ bash steps/04-verification/gates.sh
$ bash steps/04-verification/gates.sh --without-tests
$ echo $?
```

Room does

- Writes one script for their own repository with the same ending.
- Confirms three distinct exit codes for passing, failing and misconfigured.

Ask *Which of your checks is currently optional, and who decided that?*

0:30 to 1:00

Tests the writer never sees

Goal Each person has a small set of behavioural checks kept out of the implementing agent's reach.

- Say**
- Explain the circularity plainly: when the same run writes the tests and the code, green means the code agrees with itself.
 - Choose what to hold out together. The contract and the behaviour a user would notice, not the internals.
 - Head off the objection before it arrives. This is a sample, not a second suite, and a handful is enough.
-

Run

```
$ ls steps/04-verification/holdout
$ node steps/04-verification/demo.ts holdout
```

- Room does**
- Sees the holdout set run after the change, against work that never saw it.
 - Picks three behaviours in their own project worth holding out.
-

Ask *What behaviour would embarrass you most if it broke quietly? Start there.*

1:00 to 1:30

The check no gate can make

Goal Everyone has run it against a test they already trusted, and some of them are surprised.

- Say**
- Have them pick a test they wrote themselves and are confident in. The confidence is the point.
 - Walk the sequence together: undo the change it covers, run it, expect red, restore.
 - Ask who found one that stayed green. There is usually at least one, and that discovery does more teaching than the explanation.

Run

```
$ node steps/04-verification/demo.ts negative-proof
$ node steps/04-verification/demo.ts negative-proof --toothless
```

- Room does**
- Applies the sequence to one of their own tests by hand.
 - Wires it into their loop so it happens on every change rather than when remembered.

Ask *How many of your tests have never once been red? What does that number mean?*

1:30 to 2:00

Deciding what a result may claim

Goal Their pipeline downgrades a conclusion whose evidence is missing, without consulting what the agent said.

- Say**
- Separate the two jobs sharply. Investigation is where judgement helps. The sentence a person will act on gets no freedom at all.
 - Write the rules as ordinary code in front of them, reading the artifacts the run left behind.
 - Connect it to the cross-language case: a definition changed in one place, and the proof required is three suites agreeing, not one claim that it went fine.
-

Run

```
$ node steps/04-verification/demo.ts evidence
$ node steps/04-verification/demo.ts evidence --lost-artifact
```

- Room does**
- Sees a confident conclusion downgraded because its proof could not be found.
 - Writes one such rule for their own repository.
-

Ask *What does your pipeline currently believe purely because something reported it?*

1:35 to 1:50

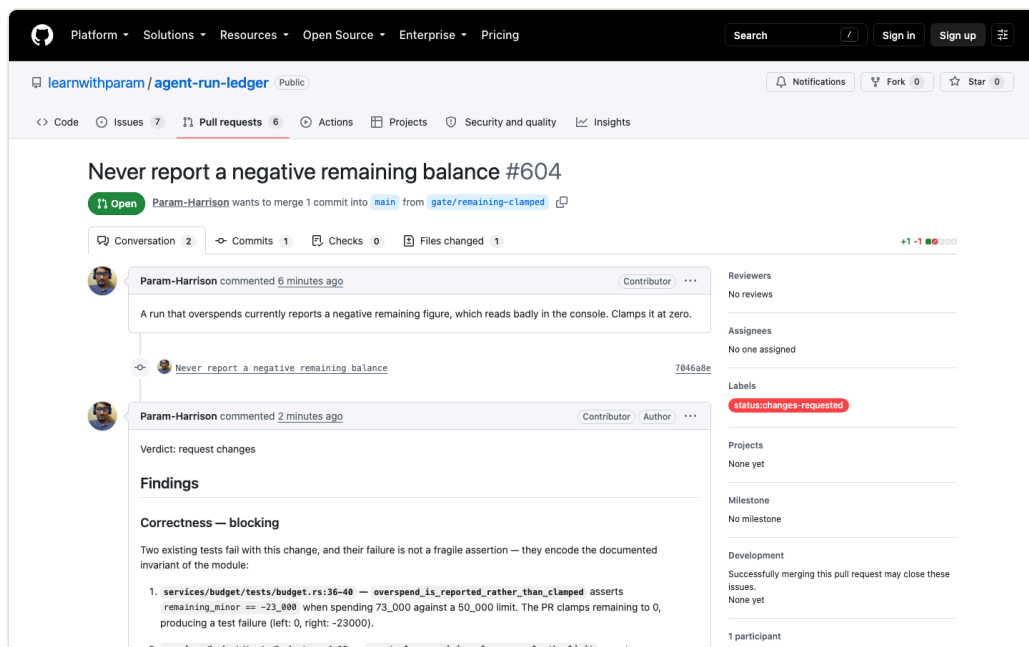
A gate failing in one language

Goal They can defend running only the checks a change reaches, and name what that misses.

- Say**
- The ownership graph decides which suites a change is measured by, so a schema edit is judged by all three.
 - Show the failure and note that the language it failed in is not the language the issue named.
 - Then name the cost honestly: a wrong graph means a real failure nobody ran.

Run

```
$ make check
```



One change, three suites, and only one of them red.

Show on the board

Work to the cross-stack item to its session to three suites, one change

Only the suites a change reaches

The ownership graph decides what runs. A schema edit is judged by all three languages; a console change is judged by one. Show which ran and which did not.

A wrong graph is a failure nobody sees

If a directory is claimed by the wrong target, its suite never runs against changes that reach it. That is the cost of this design and it belongs in the room rather than in a footnote.

The verdict is quoted, never summarised

The record carries the gate's own last line. A summary is a second source of truth about whether something passed.

Room does

- Reads which suites ran and which did not.
- Names one path in their repository whose owner is currently ambiguous.

Ask

If your graph were wrong by one directory, which failure would you never see?

Concepts: the result line, held-out checks, an independent read, proving the test bites, evidence rules, one definition.

Build Self-Healing Engineering Loops

Recovering from a failed check without a human restarting anything, and knowing when to give up. The second half is about stopping, which is the part people skip.

0:00 to 0:35

Handing back a reason, not a rejection

Goal	A failed check produces a next attempt that starts from what went wrong.
-------------	--

- | | |
|------------|---|
| Say | <ul style="list-style-type: none">• Show the useless version first: a failure returned as a status with no detail, and the retry that repeats the same mistake.• Build the useful shape together. What failed, the output, the file, and what the task said should have been true.• Be clear about why the roles are split. An agent grading its own output has already invested its reasoning in defending it. |
|------------|---|
-

Run

```
$ node steps/05-loop/demo.ts run 21
```

The loop is `steps/05-loop/loop.ts`.

- | | |
|------------------|--|
| Room does | <ul style="list-style-type: none">• Watches a first attempt fail, and reads the packet that goes back.• Watches the second attempt pass, having been told precisely what was wrong. |
|------------------|--|
-

Ask	<i>What does your pipeline hand back today when something fails, and would it help anyone?</i>
------------	--

0:35 to 1:10

Three ways to end

Goal Nobody leaves with a loop that can run indefinitely.

- Say**
- Ask what happens to their loop when the task is impossible. The honest answer is usually that nobody has tried.
 - Name the three endings and make them choose numbers for their own project rather than copying the lab's.
 - Reframe giving up. An item returned to a person with its attempts recorded is a result; a run still going is not.
-

Run

```
$ node steps/05-loop/demo.ts run 22 --impossible
```

- Room does**
- Sees attempts exhausted and the item escalated with its history intact.
 - Sets an attempt limit and a time limit in their own configuration.
-

Ask *What is the most you would spend on one item before a person should look at it?*

1:10 to 1:45

Where the time went

Goal Everyone is timing each stage separately and can see which one to attack.

Say

- Ask them to guess the split before showing it. The guesses are almost always wrong in the same direction.
- Show the recorded numbers and let the surprise do the work, then name the three usual culprits: setup repeated per attempt, checking that repeats work already done, and waiting on a person with nothing saying so.
- Give the actionable version. Reuse caches across workspaces, let the reviewer consume evidence rather than regenerate it, and make waiting visible.

Run

```
$ node steps/05-loop/demo.ts budget
```

Room does

- Reads a waterfall of stages for one completed item.
- Names the stage they would reduce first, and says how.

Ask *Which stage would you attack first on your own project, and what evidence do you have?*

1:45 to 2:00

Capstone scope

Goal Everyone has chosen the real item from their own backlog that they will demonstrate.

Say

- Give the selection rule: small enough to finish, real enough that somebody wanted it, and touching more than one area if possible.
- Warn against the demonstration-shaped item. Choosing something invented for the occasion wastes the exercise.

Run Nothing. Backlogs open, choices written in the shared document.

Room does Commits to one item by name and posts it where the group can see.

Ask *Is anyone's choice too large? Say it now rather than on demonstration day.*

1:35 to 1:50

Coming back from red

Goal They can distinguish a retry that uses the reason from one that merely repeats.

- Say**
- A loop that retries without reading the failure is a slower way to fail the same way.
 - Compare the two attempts and find the sentence in the second that only exists because of the first.
 - Cap the attempts out loud: twice with the reason attached, then a person.

Run

```
$ make score
```

Never report a negative remaining balance #604
Param-Harrison wants to merge 2 commits into [main](#) from [gate/remaining-clamped](#)

test must be revised or removed — its assertion no longer holds for overspend cases, and removing it requires justification.

2. Update the `assess()` docstring (lib.rs:98-99) to describe the clamped behavior instead of the negative-reporting behavior. Clarify why clamping is preferred over the previous explicitly-chosen design.
3. Update the module-level invariant documentation (lib.rs:5-6) to reflect that `spent + remaining == limit` holds only when spend does not exceed the limit.

Alternatively, if the existing design of negative remaining is intentional and correct (which the tests and docs assert), close this PR without merging.

Assumptions

- The author's intent is cosmetic — "reads badly in the console" — not a correctness fix. The existing `format_error` function already handles negative values, so this is a design preference, not a bug.
- The PR is a first-time contribution from this author (0 merged PRs), which warrants closer scrutiny of the design rationale.
- The absence of an issue means this design change was not discussed or approved before implementation.

Open questions

- Should the console display logic be updated instead of the money path? For example, the console could format overspend as "€0.00 remaining (overspent by €230.00)" without changing the budget engine's invariant.

Review runtime: openrouter/deepseek/deepseek-v4-flash, reasoning setting: off.

Param-Harrison added the `status: changes requested` label 6 minutes ago

Param-Harrison commented 5 minutes ago

Nothing has changed since my last review pass. The previous verdict (`request: changes`) still stands: 2 tests fail, the documented invariant is broken, and the design change lacks issue context. The three requested changes are still unaddressed.

Attempt two, beginning from the failure text rather than from the issue again.

Show on the board

Work to a card that retried to its session to attempt two

The second attempt starts from the reason

Find the sentence in attempt two that only exists because attempt one failed. If there is none, the loop is repeating rather than recovering, and it will repeat for ever.

An error can survive its own fix

A decision that succeeded on its second attempt still carries the text of the first failure. Anyone treating the presence of an error as failure will report healthy runs as broken.

Twice, then a person

The charter caps attempts and says what happens after. Ask each table for their number and what they would do with the third failure.

Room does

- Finds the failure text carried into the second attempt.
- Sets a retry limit for their own repository and writes the reason beside it.

Ask

How many times should something fail before the failure becomes yours rather than its?

Concepts: two roles, how a loop ends, the time budget.

Cohort module six, two hours

Orchestrate and Ship Verified PRs

Everything connects. The last half hour is the build against buy conversation, answered by mapping what they built onto a hosted product.

0:00 to 0:45

One workflow, six layers

Goal Each person's pieces run as a single command against a real item in their own repository.

- Say**
- Resist rebuilding anything. This session connects what exists, and the connecting code should be short and boring.
 - Walk the seams rather than the parts: what triage hands to implementation, what implementation hands to checking, what checking hands to delivery.
 - Point out that every handoff is written somewhere durable, so a run that dies can be picked up rather than restarted.
-

Run

```
$ make demo STEP=06
```

- Room does**
- Runs the whole sequence on their own item, unattended.
 - Reads their own handoff records and confirms each stage could be resumed.
-

Ask *Kill your run halfway. Can the next one continue from what it left behind?*

0:45 to 1:15

What a reviewer receives

Goal Their requests arrive with enough attached that a colleague could judge one in a few minutes.

- Say**
- Have them review each other's output rather than their own. Reviewing your own factory tells you nothing.
 - Collect what reviewers wanted and did not find, and add those to the template on the spot.
 - Be firm about the boundary. Nothing merges itself at any tier, and the enforcement belongs in the repository settings rather than in a script.
-

Run

```
$ node steps/06-delivery/demo.ts open-pr
$ node steps/06-delivery/demo.ts try-merge
```

- Room does**
- Reviews a peer's request and reports what was missing.
 - Watches an attempted merge blocked, and checks their own protection settings.
-

Ask *What did your partner need that your template did not give them?*

1:15 to 1:45

The same thing, bought

Goal Everyone can decide for their own organisation, with reasons that are not about preference.

Say

- Open the hosted product and map its stages onto their own, out loud, one at a time. The correspondence is close and that is the lesson.
- Show where their policy lands on the other substrate: a function that permits or refuses a move, holding the same rules they wrote in week one.
- Frame the decision honestly. This is about who maintains the parts you now understand, not about whether to have them.

Run

```
$ make demo STEP=07
```

The mapping is written out in `steps/07-mastra/README.md`.

Room does

- Sees a board whose columns are the stages they built by hand.
- Sees their protected path expressed as a policy function on the other substrate.

Ask *Which layers would you hand to a vendor, and which would you never give up?*

1:45 to 2:00

Demonstration day

Goal Everyone knows the shape of what they will show and how long they have.

Say

- Give the running order and the timing, and say what you will ask each of them.
- Set the expectation that a refusal or an escalation is a better demonstration than a clean pass, because it shows the limits working.

Run Nothing. Schedule on screen.

Room does Picks a slot and confirms their item still runs end to end.

Ask *Anyone whose item stopped working this week? Bring it to Friday instead of debugging alone.*

1:35 to 1:50

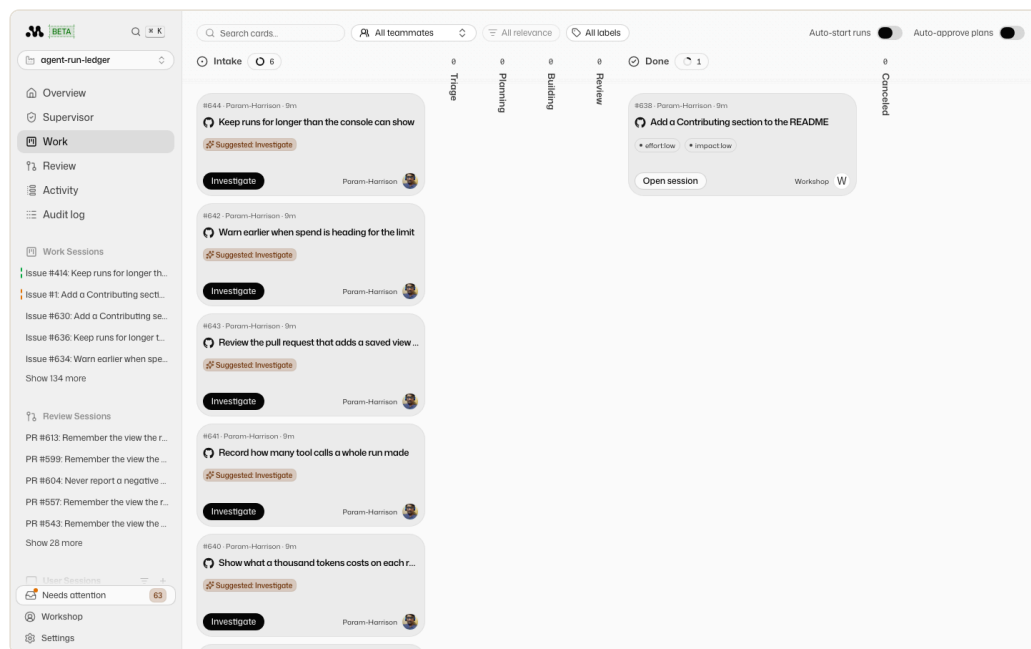
Finished, and still not merged

Goal They stop treating a board reaching done as a claim that anything shipped.

- Say**
- Done on the board means the machine has nothing left to do, which is not the same as merged.
 - Two boards, and this is the one for issues; the pull request lives on the other with its own phases.
 - Anyone who conflates them will report a week of work that is still sitting unreviewed.

Run

```
$ make e2e
```



The item reaches done while its pull request stays open.

Show on the board

Review to its own phases to the same change on Work to done, unmerged

Done means the machine is finished

The work item reaches done when its pull request exists. Nothing has merged, nobody has read it, and the board is not claiming otherwise.

The same change, two boards

Find it on Work as an issue and on Review as a pull request. They have different phases because they are different questions: is this worth doing, and is this done well.

A re-review needs a push

The review board has no review-to-review step. Fixing the branch is what starts the second pass, and asking the board to review again does nothing at all.

Room does

- Finds the same change represented on both boards.
- Writes the sentence they would use in a status update, and has it corrected.

Ask

Your dashboard says twelve done. How many of those can a customer use?

Concepts: durable state, the delivered request, who decides, the record, stages on a board, policy that stays yours.

Design an AI-Native Software Factory for Your Organization

A working session for people who decide how engineering works. They do not write code here. They leave with an operating model, an autonomy matrix and a sequenced roadmap.

0:00 to 1:15

What you already have, unwritten

Goal	Each team's current rules are on paper, and the variation between teams is visible to everyone in the room.
Say	• Open by asserting that they already have an operating model. It is informal, it differs by team, and nobody has compared the versions. • Have each team write theirs on a card: what a machine may do unattended, what needs approval, and who grants exceptions. • Put the cards side by side and let the spread speak. That gap is the reason for the day, and it is more persuasive than any slide.
Run	Nothing technical. Cards on a wall.
Room does	• Produces one card per team and reads two others aloud. • Marks every place two teams contradict each other.
Ask	<i>Where two of your teams disagree, which one is right, and who would decide that today?</i>

1:30 to 3:00

The autonomy matrix

Goal Their estate is sorted by consequence, with approval placed where the risk actually is.

- Say**
- Kill the difficulty axis early, using an example from their own estate rather than a generic one.
 - Run the sort as a group exercise across their real services, and let the arguments happen. The arguments are the product.
 - Push back whenever everything lands in the strictest column. A model where nothing may proceed is a model nobody will follow.
-

Run

```
$ node steps/01-boundary/demo.ts graph
```

Show the worked example once, as a shape to copy, then put it away.

- Room does**
- Places every significant service in a column, with a named owner.
 - Agrees who may move something between columns and how that gets recorded.
-

Ask *Which service caused the longest argument? That is the one your written policy most needs to cover.*

3:45 to 5:00

Protecting the standard you already have

Goal They know which checks must run on every change and which belong before release, and where their current setup fails open.

- Say**
- Ask what a machine could produce today that would pass their pipeline and still be wrong. Let the room be uncomfortable.
 - Introduce failing closed as a governance property rather than a technical one, because a missing check is invisible in every report they receive.
 - Explain the independent read in terms they own: an assurance step, with a cost per change they can decide to pay.
-

Run

```
$ bash steps/04-verification/gates.sh --without-tests
```

- Room does**
- Sees a missing check produce something that is neither a pass nor a failure.
 - Lists the checks that will be required across every team.
-

Ask *How would you learn, this month, that one of your teams quietly disabled a required check?*

5:00 to 5:30

Who is accountable for a change nobody wrote

Goal Accountability is assigned explicitly, and the capacity limit is a number rather than a feeling.

- Say**
- Put the question bluntly, because it is the one their board will ask them. A machine wrote it, someone approved it, and something went wrong.
 - Settle it the same way as any other change: a named engineer approved it and owns it.
 - Then give them the constraint that follows. Approval capacity is finite, so the pipeline must stop producing when it is full.
-

Run

```
$ node steps/01-boundary/demo.ts capacity
```

- Room does**
- Writes the accountability rule in one sentence.
 - Sets a review capacity per team and says how it will be measured.
-

Ask *How many changes per week can each of your teams genuinely stand behind?*

5:30 to 6:00

Sequencing it

Goal	They leave with an ordered plan whose first step is small enough to happen next week.
-------------	---

- | | |
|------------|--|
| Say | <ul style="list-style-type: none">• Insist the first move is boring and cheap. Trust is built by a small thing that works, not by a programme.• Have them pick the first workflow by two criteria: it happens often, and getting it wrong is recoverable.• Name what they will measure before starting, so the second conversation is about numbers rather than impressions. |
|------------|--|
-

Run	Nothing. The roadmap template on screen.
------------	--

- | | |
|------------------|--|
| Room does | <ul style="list-style-type: none">• Fills in the first three steps with owners and dates.• Agrees the baseline they will compare against later. |
|------------------|--|
-

Ask	<i>What will you be able to show in six weeks, and who needs to see it?</i>
------------	---

2:20 to 2:40

Four shapes, one afternoon

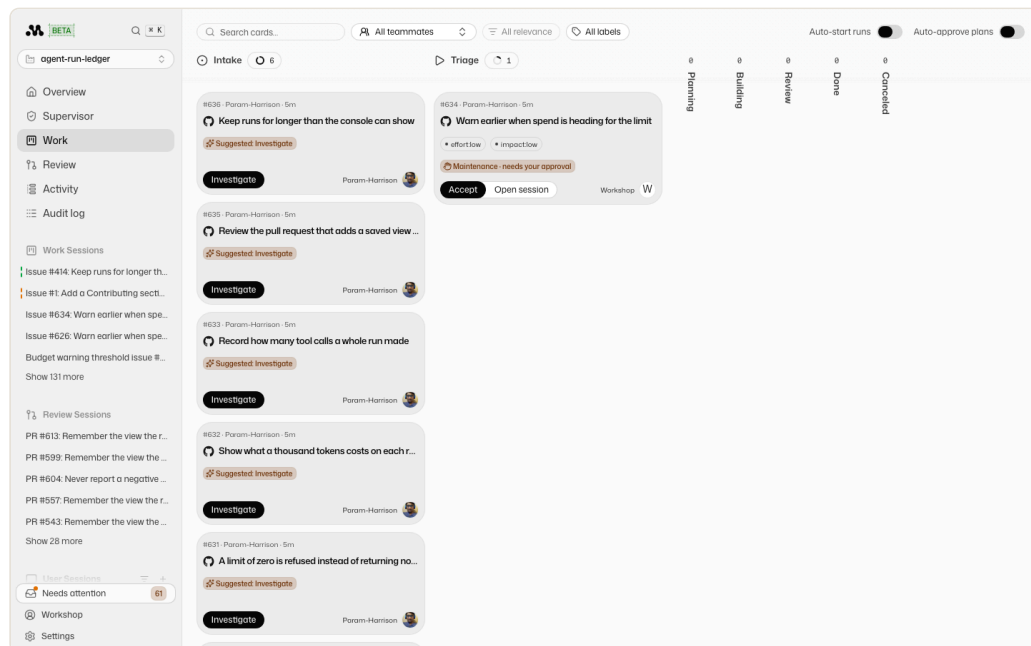
Goal Each leader has chosen a shape for one real team and can defend the cost it imposes.

Say

- Switch the shape live and re-run the same item so the room sees the rules doing the work rather than the model.
- None of the four is correct. Each is a position about who carries the cost of being wrong.
- Push back on anyone choosing the strictest by default: it has a price, and it is paid in throughput by people who are not in this room.

Run

```
$ make profile NAME=solo
$ make profile NAME=enterprise
```



The same item, under the strictest of the four shapes.

Show on the board

Settings to the two switches to Work to the money item

A shape is a commit, not a setting

make profile writes the charter and the ownership graph and pushes them, because a run clones from the remote. A shape that exists only on somebody's laptop is a shape no agent will ever read.

The queue cap is the number to argue about

Every shape declares how many decisions may wait on a person. That number, not the model, is what decides whether the factory helps or buries the team.

Nobody in the room pays the cost of the strictest shape

Pushing everything to refuse feels responsible and is paid for in throughput by people who are not at this table. Ask who they are and whether they agreed.

Room does

- Picks a shape for one named team and writes the trade-off it accepts.
- Names the first rule they would relax after a quarter of evidence.

Ask

Who in your organisation currently pays for a change being wrong, and do they know?

Concepts: the six jobs, a written policy, grading autonomy, paths off limits, capacity, accountability, measuring, ownership.

Evidence to bring: what adoption measured, felt against measured speed, the named risk classes, routing as published practice.

Weekly hour, plus the final session

Office hours and capstone demo day

Unscripted by design. This sheet exists so the hour has a shape when nobody brings anything, and so the last one runs to time.

First ten minutes

Collect before solving

Goal	Every blocker in the room is written down before any of them is worked on.
Say	• Ask for problems in one line each and write them all up before touching any of them. • Look for the repeated one. When three people are stuck the same way, that is a teaching gap rather than three bugs. • Say the order out loud so nobody is waiting without knowing whether their turn is coming.
Run	Nothing. A list on screen.
Room does	Contributes one line each, including those who came only to listen.
Ask	<i>Anything here that more than one of you recognises?</i>

As they come up

The four that always appear

Goal Recurring problems get a fast answer so the hour goes to the interesting ones.

- Say**
- A new workspace with no dependencies installed. Answer with caching and a preparation command rather than a longer setup step.
 - Context that never arrives. Almost always a path the ownership graph does not claim, so run the graph check first.
 - A check that passes when it should not. Apply the reverting proof before debugging anything else.
 - A run that never ends. Look for the missing limit rather than the slow model.
-

Run

```
$ node steps/02-execution/demo.ts selftest
$ node steps/01-boundary/demo.ts graph
```

Room does Runs the same two commands on their own project before describing the symptom.

Ask *What did the self-check say? Read it out before we theorise.*

Final session only

Demonstrations

Goal Everyone shows their own item moving, and gets one useful piece of feedback from the group.

- Say**
- Give each person a fixed slot and hold it, so the last presenter gets the same time as the first.
 - Ask the same two questions of everyone: which layer took the longest to get right, and what did it refuse to do.
 - Direct the feedback towards what the group can use rather than towards praise.
-

Run Their machines, their repositories. Yours stays closed.

- Room does**
- Shows an item becoming a reviewed request on their own codebase.
 - Names one thing they would build differently.
-

Ask *What will you take back to your team on Monday, in one sentence?*

0:20 to 0:40

Reading somebody else's evidence

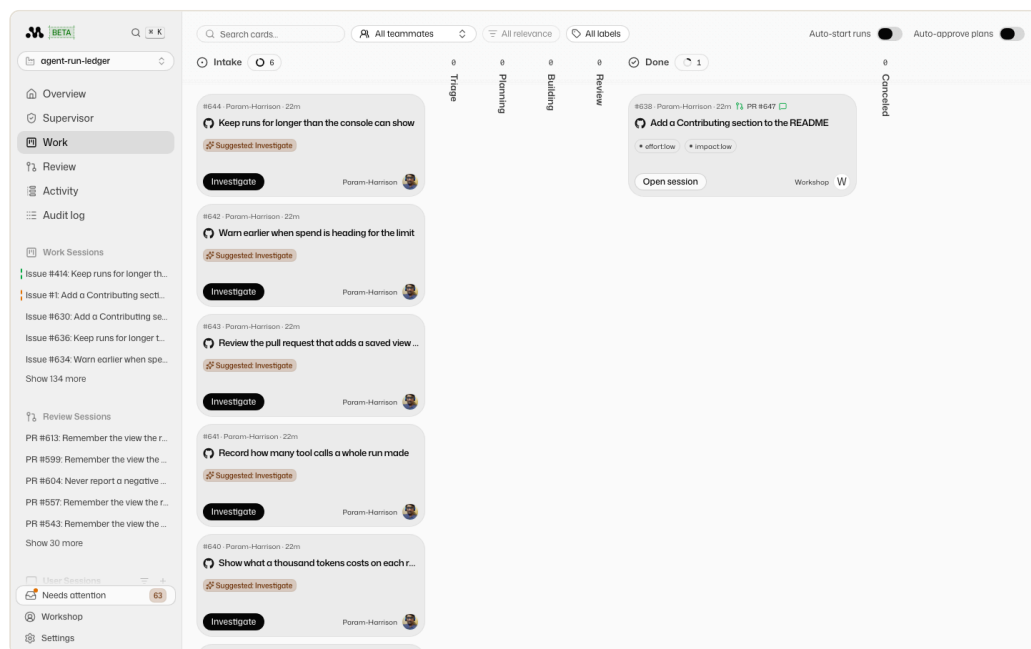
Goal Everyone has read one run they did not perform and formed a view without being told the story.

Say

- Bring up a capstone board and review it cold, with no narration from its author.
- Judge the evidence rather than the outcome, because a green run with no proof is the thing this whole course argues against.
- Let the author respond only after the room has committed to a verdict.

Run

```
$ make status
$ make score
```



Pull requests on their own board, with their own phases.

Show on the board

Review to somebody else's board to a pull request to the verdict

Judge the evidence, not the outcome

A green run with no proof is the thing this course argues against. Ask what would have to be true for this verdict to be wrong, before the author says anything.

The author speaks last

Let the room commit to a view first. A narrated demo teaches the narrator, and everyone else watches somebody else be competent.

One number each, and what it cost

Every capstone reports a coverage figure and a score. Ask which of the two they trust and why, because the gap between them is the only interesting part.

Room does

- States a verdict on a run they did not make.
- Names the one piece of evidence that would have changed it.

Ask

What did you believe about this run before the author spoke, and what changed after?

Concepts: records, endings.